

小念

XiaoNian

你电脑里会成长的生活助理

本地优先 · 隐私安全 · 越用越懂你 · 主动关心 · 帮你干活 · 按需进化

(͡° ͜ʖ ͡°) っ

“AI 不该只是回答问题的工具，而应是贯穿你生活、主动提供帮助的伴侣。”

小念把这句愿景，做成一个真正能在你电脑上把事办成、且只属于你自己的私人秘书。

关键地址（评申请直接访问）

项目介绍主页	http://47.119.112.225/xiaonian/
在线 Demo	http://47.119.112.225/xiaonian/app/
GitHub 源码	https://github.com/bcefgjhj/xiaonian
项目导航（多项目）	http://47.119.112.225/

参赛者信息

参赛者	戴尚好	学校	中国科学技术大学
GitHub	github.com/bcefgjhj	邮箱	bcefgjhj@163.com
个人主页	bcefgjhj.github.io	日期	2026 年 6 月

摘要 · Executive Summary

一句话介绍

小念 (XiaoNian) 是一个本地优先、隐私安全的个人生活助理 / AI 秘书：住在你电脑里，越用越懂你、会主动关心你、能帮你在电脑上把活干完，并且可以按需安装 Skill 自我进化。核心只保留四件事（懂你 / 主动关心 / 干活 / 进化），一切”杂”能力下沉为可插拔技能——功能不杂，但能无限生长。

为什么是”企业级”，而不是玩具

底层全部站在大厂在生产验证过的成熟开源框架上（以依赖方式引入），我们只写产品逻辑：编排用 **LangGraph** (Klarna / Uber / 摩根大通生产使用)、记忆用 **Mem0** (业界领先 Agent 记忆层)、技能用 **Anthropic Agent Skills 开放标准**、模型层多 provider 可切换 (**MiniMax M2** / **DeepSeek** / 小米 **MiMo**, OpenAI 与 Anthropic 双协议)。配套场景评测、可观测 / 成本追踪、审计日志、架构决策记录 (ADR) 与三种部署模式。

当前完成度（可运行的成品，而非 PPT）

全栈已落地并端到端跑通,且已公网部署到阿里云服务器:本地 / 服务器 daemon + LangGraph 编排 + Mem0 记忆 + 电脑操控 (含两步确认) + Skill 系统 (3 个示范技能) + 主动关心引擎 + 宠物形象 + Web UI + 场景评测。评测 **8/8 场景 100% 通过**；写文件等危险操作**实测触发两步确认并真实落地**；线上 Demo 已接入**真实 MiniMax M2 大模型** (Anthropic 兼容端点)。

核心指标速览

一眼看懂小念

- 4** 核心能力（懂你 / 主动关心 / 干活 / 进化），精简稳定
- ∞** 可插拔技能（Agent Skills 开放标准），能力无限生长
- 100%** 关键数据（记忆 / 画像 / 审计 / 凭证）默认留在本机
- 8 / 8** 真实场景评测全部通过
- 2** 模型协议（OpenAI 兼容 + Anthropic 兼容），多 provider 热切换
- 3** 部署模式（本地个人版 / 服务器企业版 / 跨厂商生态版）

本规划书结构导读

全文共 4 个部分 21 章 + 8 个附录：第一部分（第 1-4 章）讲团队、主题愿景、需求洞察与产品哲学；第二部分（第 5-10 章）讲总体架构与六大技术内核（编排 / 模型 / 记忆 / 操控 / 技能）；第三部分（第 11-17 章）讲主动关心、宠物、安全隐私、可观测、Web UI、部署运维与评测体系；第四部分（第 18-21 章）讲工程实践、行业背书、商业化蓝图与小米生态契合。附录给出关键代码清单、API 文档、配置说明、评测明细、技能样例与安装指南。

目录

摘要 · Executive Summary	1
第一章 团队介绍	1
1.1 参赛者	1
1.2 分工与职责	1
1.3 工作方法论	2
第二章 作品主题与愿景	3
2.1 主题：把”手机助理”做成”生活助理”	3
2.2 愿景：Life OS——贯穿一生、主动提供帮助的伴侣	3
2.3 命名故事：为什么叫”小念”	3
2.4 设计目标（可度量）	4
第三章 需求洞察	5
3.1 现状：助理还没真正成为”助理”	5
3.2 四大痛点	5
3.3 趋势：从”偶发问答”到 Life OS	6
3.4 目标用户与场景	6
第四章 产品设计哲学	7
4.1 四件核心 + 无限技能	7
4.2 关键取舍	7
4.3 设计原则	8
4.4 一天的故事线（演示脚本）	8
第五章 总体架构	9
5.1 设计目标驱动的分层架构	9
5.2 一次对话的端到端数据流	9
5.3 组件清单	10
5.4 逐层选型与背书	11
第六章 编排内核：LangGraph 状态机	12
6.1 为什么是 LangGraph	12
6.2 状态机拓扑	12
6.3 状态定义与节点实现	12

6.4 多轮控制与防失控	13
第七章 模型内核：多 Provider 与双协议	15
7.1 统一抽象，多家可换	15
7.2 关键工程：MiniMax M2 的 Anthropic 协议适配	15
7.3 成本与延迟观测	16
7.4 优雅降级：离线 mock	16
第八章 记忆内核：“懂你”	17
8.1 Mem0 + 本地隐私嵌入器 + 结构化画像	17
8.2 本地隐私嵌入器	17
8.3 结构化画像（强类型事实）	18
8.4 记忆生命周期	18
第九章 执行内核：“干活”	19
9.1 受控 harness：读即时，写确认	19
9.2 两步确认时序	19
9.3 可选高级后端：Open Interpreter	20
第十章 进化内核：“按需进化”	21
10.1 Agent Skills 开放标准	21
10.2 技能注册表	21
10.3 三个示范技能	22
10.4 编写一个新技能：三步	22
第十一章 主动关心引擎	23
11.1 从“被动应答”到“主动关心”	23
11.2 生活状态推断（LifeState）	23
11.3 静默协议：克制是一种修养	23
第十二章 宠物形象：情感化的“脸”	25
12.1 把状态变成一张会变的脸	25
第十三章 安全与隐私链（核心卖点）	26
13.1 威胁模型	26
13.2 PII 上云前脱敏	26
13.3 本地审计日志	27
第十四章 可观测性	28
14.1 每一次调用都可追溯	28
14.2 成本估算	28

第十五章 Web UI 设计	29
15.1 零框架、三栏布局	29
15.2 真机截图：在线 Demo	29
15.3 两步确认弹窗	30
15.4 流式输出（SSE）	31
第十六章 部署与运维	32
16.1 三种部署模式	32
16.2 已落地：阿里云公网部署	32
16.3 多项目管理策略	32
16.4 项目导航页（多项目入口）	33
16.5 项目介绍页	34
第十七章 评测体系	37
17.1 借鉴 VitaBench：看”端到端完成率”	37
17.2 8 个场景与结果	37
17.3 评测指标定义	38
第十八章 工程实践	39
18.1 目录结构	39
18.2 架构决策记录（ADR）	39
18.3 配置与依赖管理	40
18.4 优雅降级的工程价值	40
第十九章 行业背书与对标	41
19.1 每个技术点都有出处	41
19.2 与豆包的正面对比	41
第二十章 商业化与未来展望	42
20.1 产品路线图	42
20.2 四条商业化路径	42
20.3 商业模式画布（简版）	42
20.4 风险与应对	43
第二十一章 与小米生态的契合 & 总结	44
21.1 为什么适配小米生态	44
21.2 总结	44
附录 A 关键代码清单	46
A.1 启动入口与配置	46
A.1.1 run.py	46
A.1.2 config.py	46
A.2 模型层：多 Provider 路由（OpenAI / Anthropic 双协议）	48

A.2.1	model/provider.py	48
A.3	编排层: LangGraph 状态机	56
A.3.1	graph/persona.py	56
A.3.2	graph/tools.py	56
A.3.3	graph/build.py	59
A.4	记忆层: ”懂你”	63
A.4.1	memory/embedder.py (本地隐私嵌入器)	63
A.4.2	memory/store.py (Mem0 + SQLite 画像)	64
A.5	执行层: ”干活”	68
A.5.1	capabilities/computer/executor.py (读即时 / 写确认)	68
A.6	进化层: Agent Skills	72
A.6.1	capabilities/skills/registry.py	72
A.7	宠物形象	74
A.7.1	capabilities/pet/state.py	74
A.8	主动关心引擎	77
A.8.1	proactive/engine.py	77
A.8.2	daemon/life_state.py	80
A.9	安全链	81
A.9.1	security/confirm.py (两步确认)	81
A.9.2	security/redact.py (PII 脱敏)	83
A.9.3	security/audit.py (审计日志)	83
A.10	可观测性	84
A.10.1	observability/tracing.py	84
A.11	服务装配	85
A.11.1	daemon/main.py (FastAPI 应用)	85
附录 B	API 接口文档	91
附录 C	配置项说明	92
附录 D	评测场景全文	93
D.1	eval/scenarios.yaml	93
D.2	eval/runner.py	94
附录 E	技能样例全文	96
E.1	order-food / SKILL.md	96
E.2	hail-ride / SKILL.md	97
E.3	weekly-report / SKILL.md	97
附录 F	安装与快速开始	99
F.1	本地运行	99
F.2	服务器部署 (多项目)	99

附录 G 术语表	100
附录 H 参考资料	101

团队介绍

1.1 参赛者

本作品由参赛同学**独立完成**从产品定位、市场调研、架构设计，到全栈实现、公网部署、评测与文档的端到端工作。

参赛者	戴尚好
学校	中国科学技术大学
GitHub	https://github.com/bcefgjhj
邮箱	bcefgjhj@163.com
个人主页	https://bcefgjhj.github.io

1.2 分工与职责

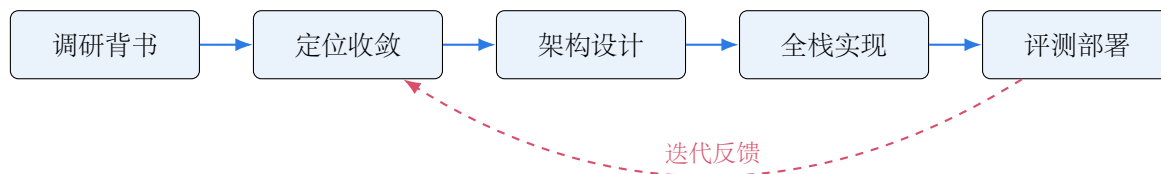
虽为个人作品，但严格按”产品—工程—质量”三条线推进，确保达到企业级交付标准：

- **产品与架构**：完成定位收敛（从最初的”情感陪伴”升级为”生活 OS 式助理”）、技术选型与背书调研（精读美团技术报告、Anthropic / LangChain / Mem0 等一手资料）、系统分层架构设计。
- **工程实现**：本地 / 服务器 daemon、LangGraph 编排、Mem0 记忆接入、电脑操控 harness、Skill 系统、主动关心引擎、安全链、多 provider 模型层（含 MiniMax M2 Anthropic 协议适配）、Web UI。
- **质量与交付**：场景评测套件、可观测 / 成本追踪、ADR 文档、阿里云公网部署（nginx + systemd 多项目）、本规划书。

1.3 工作方法论

借鉴架构，原创实现

本项目坚持”站在巨人肩膀上、但不复制源码”：所有底层框架以**依赖方式**引入（pip 安装），我们只编写产品逻辑与适配层；架构思想借鉴大厂成熟方案，但每一行业务代码均为原创。这样既保证了企业级的工程底座，又避免了直接照搬他人代码的合规风险。



作品主题与愿景

2.1 主题：把”手机助理”做成”生活助理”

现有手机 / 家居助理（小爱同学、小艺、YOYO、Siri 等）大多停留在”开灯关灯、问答式”的**工具**层面：它们能听懂一句话，却记不住你是谁；能回答一个问题，却办不成一件事；越贴近生活就越让人担心隐私。

小念要回答的问题是：如果 AI 真的是一个”住在你电脑里的私人秘书”，它应该长什么样？

2.2 愿景：Life OS——贯穿一生、主动提供帮助的伴侣

人对 AI 的使用方式，正在从”偶发性的单一任务”转向”持续性的智能交互”。AI 不再只是回答问题的工具，而应是**贯穿用户一生、主动提供帮助的智能伴侣**。其核心是**长期记忆乃至终身记忆**：用系统级的记忆、推理与跨端协作，把用户生活与工作的所有碎片，编织成持续、主动、个性化的体验。

对标与差异

对标豆包”把活干完”的办公任务模式（VibeWorking 思路：理解目标 → 自主拆解 → 调用本地电脑 / 文档 / 网页 → 一路执行到底），但小念把**隐私主权**作为核心差异：**数据留在你自己的电脑**，直接调用你本机的算力干活。

2.3 命名故事：为什么叫”小念”

名字经历了多轮收敛，最终定为”小念”：

- **两个字、亲切**：符合”陪伴型”产品的气质，朗朗上口。
- **”念”——惦念你**：核心卖点是**主动关心**，”念”恰好表达”它一直惦记着你、在意你的状态”。
- **”念”——记得你**：呼应”长期记忆”——念念不忘，必有回响；它记得你说过的每一件小事。

2.4 设计目标（可度量）

目标	可度量标准
懂你	跨会话记住画像 / 习惯 / 偏好；可在 UI 查看与删除（隐私主权）
主动	在对的时间主动发起关心 / 提醒，而非 100% 被动
干活	能在本机真实完成文件 / 命令 / 写作类任务（端到端完成率，而非模型分）
进化	新增能力 = 装一个 Skill，核心代码零改动
隐私	关键数据默认不出本机；上云内容经 PII 脱敏

需求洞察

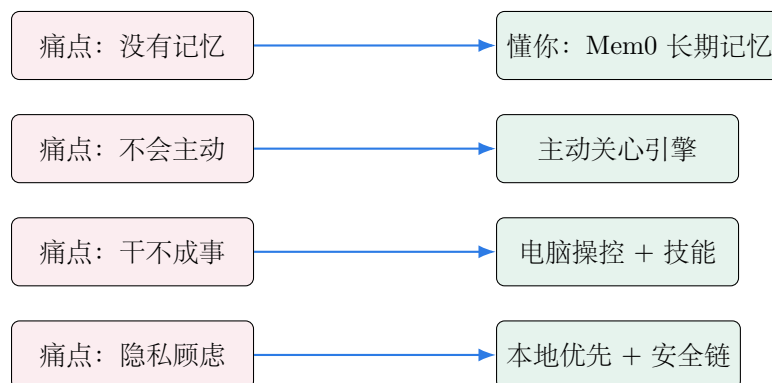
3.1 现状：助理还没真正成为”助理”

我们把市面上主流的”助理类”产品按四个关键维度（记忆 / 主动 / 干活 / 隐私）做了横向对比。结论是：**没有一个产品在”记忆 + 主动 + 本机干活 + 隐私”上同时做好**，这正是小念的机会窗口。

产品	长期记忆	主动关心	本机干活	隐私本地	定位
小爱同学 / 米家	✗	弱	✗	✗	家居语音控制
华为小艺	弱	弱	部分	✗	手机系统助理
荣耀 YOYO	弱	部分	部分	✗	手机系统助理
苹果 Siri	弱	✗	弱	部分	手机系统助理
豆包（专业版）	部分	✗	✓	✗	All-in-One 生产力
ChatGPT Memory	✓	✗	弱	✗	通用对话
小念	✓	✓	✓	✓	本地生活助理 / 秘书

3.2 四大痛点

1. **没有记忆**：每次从零开始，不懂你是谁、有什么习惯与偏好；多轮之后又”失忆”。
2. **不会主动**：只能被动等指令，不会在对的时间主动关心、提醒、查漏补缺。
3. **干不成事**：能回答，但不能在你的电脑上把一件事**完整地办完**（找文件 → 处理 → 产出）。
4. **隐私顾虑**：越贴近生活越敏感，云端助手让人不放心把全部生活数据交出去。



3.3 趋势：从”偶发问答”到 Life OS

Altman 等业界领袖提出的愿景中，AI 正成为”贯穿用户一生、主动提供帮助的智能伴侣”。落地信号包括：ChatGPT 自 2025 年起的 Memory 功能可横跨全部历史对话、保留写作风格与长期 OKR，并支持逐条查看与删除以确保隐私主权；豆包专业版推出”办公任务模式”，从”回答问题”走向”把活干完”。这说明：**记忆 + 主动 + 执行力 + 隐私主权**，正是下一代个人 AI 的胜负手。

3.4 目标用户与场景

用户画像	典型场景
个人知识工作者	需要一个记得自己、主动提醒、能直接在电脑上处理琐事与工作的”私人秘书”
学生 / 研究者	资料整理、周报 / 总结、日常提醒、状态关心
(未来) 小团队	团队秘书：任务跟进、信息沉淀、对外协作（企业版）

需求结论

市场缺的不是”更聪明的问答”，而是”真正长在你生活里、且只属于你自己的执行型伴侣”。小念用”四件核心 + 无限技能 + 本地优先”的组合，精准切入这一空白。

产品设计哲学

4.1 四件核心 + 无限技能

小念的产品结构是一个”**稳定内核 + 可生长外壳**”：内核永远只有四件事，保证精简、稳定、可靠；一切”杂”能力都做成可插拔的 Skill，让能力无限生长而内核不臃肿。

能力	说明	底座
懂你	长期记忆：用户画像 / 生活状态 / 习惯作息 / 重要关系，越用越懂你	Mem0 + 本地隐私嵌入器
主动关心	在对的时间主动问候、提醒、关心，而非被动等指令	APScheduler + 预测触发 + 静默协议
干活	真正在你电脑上把活干完：文件 / 命令 / 写作 / 浏览器	受控 harness (可选 Open Interpreter)
进化	按需装技能扩展能力：外卖 / 打车 / 周报 / 天气...	Anthropic Agent Skills 开放标准

4.2 关键取舍

做减法，比做加法更难

明确**砍掉**脱离实际、算力重、收益低的功能：

- **✗ 会议管理 / 会议转写**：并非人人需要，且转写对算力要求高、体验脆弱。
- **✗ always-on 录音 / 全程监听**：隐私风险高，与”隐私优先”定位冲突。
- **✗ 养成类宠物游戏**：宠物只做”情感化的脸”，不喧宾夺主。

把省下的复杂度，全部投入到”把四件核心做精”。

4.3 设计原则

1. **本地优先 (Local-first)**: 默认所有数据留在本机, 云只做”可选的大脑增强”。
2. **框架优先 (Framework-first)**: 能用大厂成熟框架就不自研, 自己只写产品逻辑。
3. **安全默认开 (Secure-by-default)**: 写操作默认需要二次确认, PII 默认脱敏。
4. **优雅降级 (Graceful degradation)**: 无模型额度 / 断网时, 核心链路仍可演示与使用。
5. **可观测 (Observable)**: 每一次模型调用、工具执行、确认与关心都可追溯。

4.4 一天的故事线 (演示脚本)

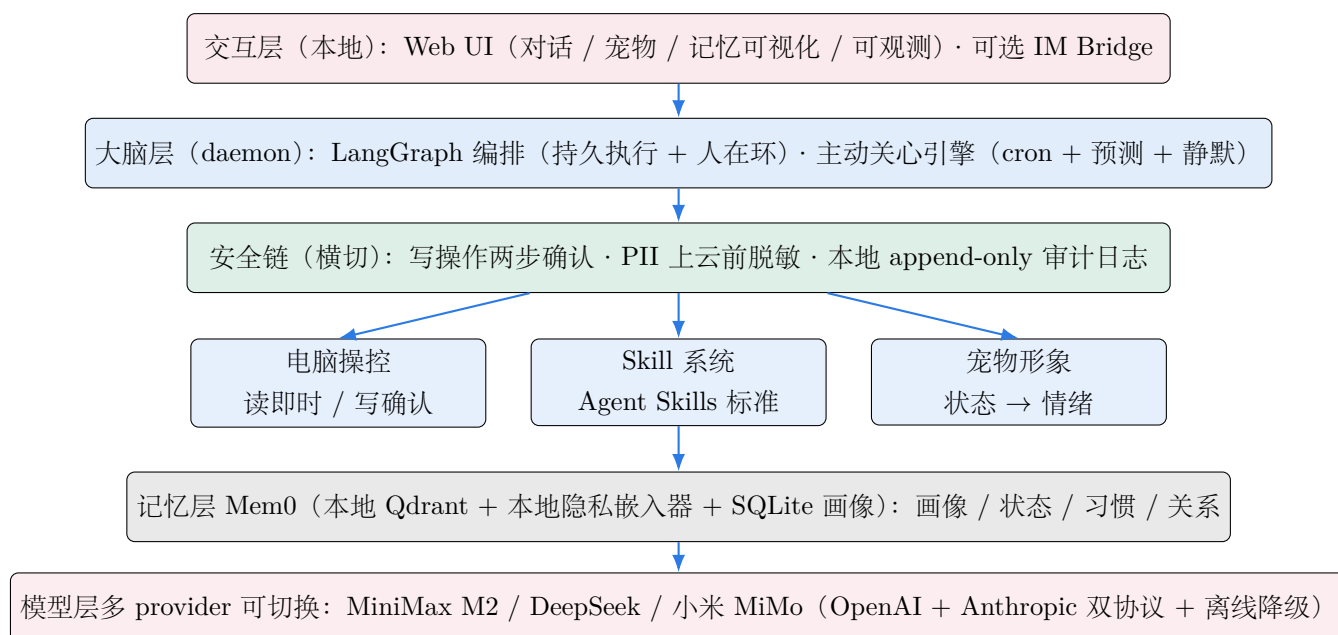


这条故事线不是 PPT 设想, 而是当前 **Demo** 中**真实可复现**的交互 (见第 15、17 章与附录评测明细)。

总体架构

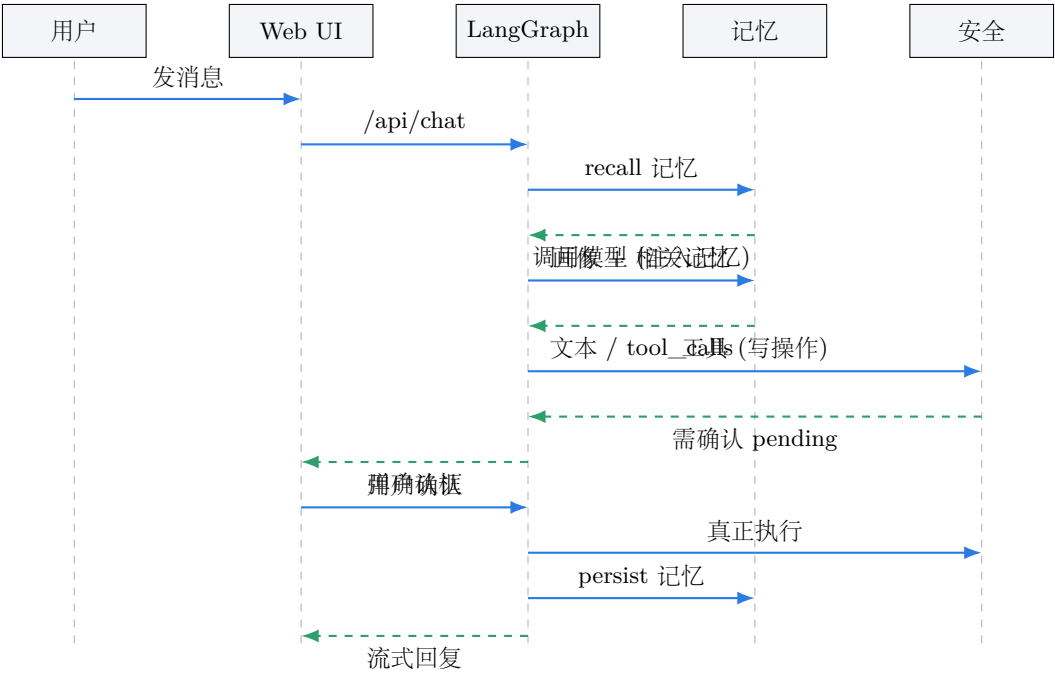
5.1 设计目标驱动的分层架构

小念采用清晰的**分层架构**：自上而下分为交互层、大脑层、安全链、能力层、记忆层、模型层。每一层职责单一、可独立替换，安全链**横切**在大脑与能力之间，确保任何”会改变世界”的操作都先过确认。



5.2 一次对话的端到端数据流

下图是用户发出一条消息后，系统内部的端到端时序（含工具调用与两步确认分支）。



5.3 组件清单

模块	职责
daemon/	FastAPI 应用：REST / SSE 接口、静态 UI、生活状态推断、装配各模块
graph/	LangGraph 状态机：persona、工具 schema 与分发、build 编排
model/	多 provider 模型路由（OpenAI / Anthropic 协议）+ 离线 mock + 成本估算
memory/	Mem0 封装 + 本地隐私嵌入器 + SQLite 结构化画像
capabilities/computer/	电脑操控 harness（读即时 / 写确认）
capabilities/skills/	Agent Skills 注册表 + 内置技能
capabilities/pet/	宠物状态：信号 → 情绪 / 表情 / 文案
proactive/	主动关心引擎（APScheduler + 预测触发 + 静默时段）
security/	两步确认 / PII 脱敏 / 审计日志
observability/	本地 trace / token / 成本追踪
eval/	场景评测套件（scenarios.yaml + runner）
web/static/	三栏 Web UI（HTML / CSS / JS，零框架）

5.4 逐层选型与背书

层	选型	依据 / 背书
编排	LangGraph	需要精确控制 + 可调试 + 生产级持久化 + 人在环; Klarna / Uber / 摩根大通生产使用, 相比 CrewAI 控制更强。
记忆	Mem0	业界领先 Agent 记忆层, 亚秒级、跨框架可移植、可本地部署。
技能	Agent Skills	Anthropic 2025 开源开放标准; 三级渐进式披露, 启动开销极低; Atlassian / Figma / Canva / Stripe / Notion 采用。
电脑操控	受控 harness (可选 Open Interpreter)	读即时、写两步确认; 可切 Open Interpreter (Apache-2.0, 原生沙箱 + 权限 + MCP)。
确认范式	人在环两步确认	借鉴美团 DPT-Agent; preview → 确认 → 执行。
评测	场景成功率	借鉴美团 VitaBench 真实生活场景思路, 看端到端完成率而非纯模型分。
模型	多 provider	UI 一键切换; MiniMax M2 主力 (1M 上下文、原生 Function Calling、多模态), DeepSeek 备选, MiMo 生态加分。

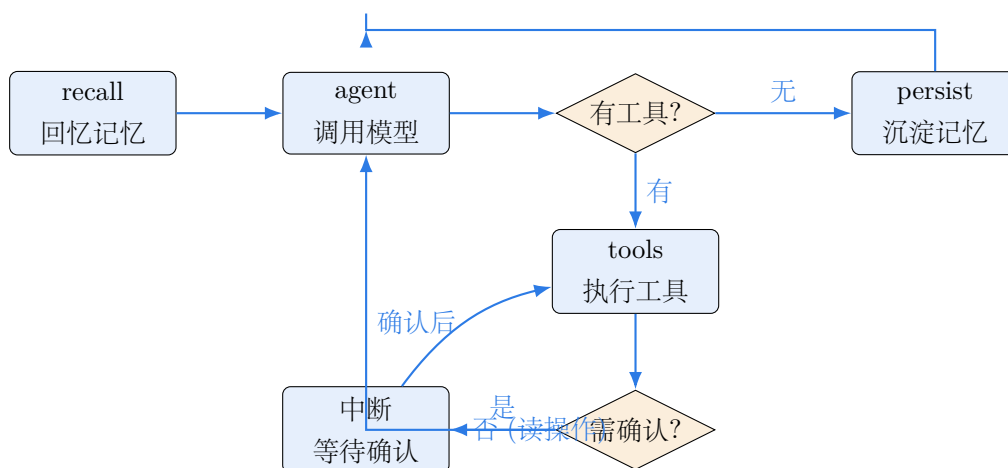
编排内核：LangGraph 状态机

6.1 为什么是 LangGraph

一个“会干活”的 Agent 不是简单的“模型 + while 循环”。它需要：多轮工具调用、**在危险操作处中断等人确认**、出错可恢复、状态可持久化、可观测可调试。这正是 LangGraph 的强项——它把 Agent 表达为一张**有状态的图**，节点是计算、边是路由，天然支持中断 / 恢复（human-in-the-loop）与检查点持久化。

6.2 状态机拓扑

小念的主循环是一张五节点图：



路由规则：`recall` 注入画像与相关记忆 → `agent` 让模型决策 → 若返回 `tool_calls` 则进 `tools`；写操作在此**中断**并向 UI 抛出确认请求，确认后再真正执行；读操作直接执行并回到 `agent`；无工具时进 `persist` 沉淀本轮记忆并结束。

6.3 状态定义与节点实现

Listing 6.1: AgentState 与核心节点 (graph/build.py 摘选)

```

1 class AgentState(TypedDict, total=False):
2     messages: List[Dict[str, Any]] # OpenAI 风格消息历史
3     user_text: str # 本轮用户输入
4     reply: str # 最终回复
5     tool_events: List[Dict[str, Any]] # 工具调用事件 (喂给 UI 可视化)
6     pending_confirm: Optional[Dict] # 待确认的写操作
7     rounds: int # 工具轮次 (防失控)
8
9 def _recall_node(state):
10     # 取结构化画像 + 与本轮输入相关的记忆, 拼成 system 注入
11     profile = memory.profile_summary(USER_ID)
12     related = memory.search(state["user_text"], USER_ID, limit=5)
13     sys = build_system_prompt(profile, related)
14     msgs = [{"role": "system", "content": sys}] + state["messages"]
15     return {"messages": msgs, "tool_events": state.get("tool_events", [])}
16
17 def _agent_node(state):
18     result = router.chat(state["messages"], tools=TOOL_SCHEMA)
19     msg = {"role": "assistant", "content": result.text}
20     if result.tool_calls:
21         msg["tool_calls"] = to_openai_tool_calls(result.tool_calls)
22     return {"messages": state["messages"] + [msg],
23           "reply": result.text, "rounds": state.get("rounds", 0)}

```

6.4 多轮控制与防失控

工具轮次 `rounds` 设上限 (默认 4 轮), 超过则强制收口, 避免模型陷入 “调用—再调用” 的死循环; 每一步的工具事件都写入 `tool_events`, 既用于 UI 的 “工具调用” 可视化, 也写入审计日志。

Listing 6.2: 路由与人在环中断 (graph/build.py 摘选)

```

1 def _route_after_agent(state):
2     last = state["messages"][-1]
3     if last.get("tool_calls") and state.get("rounds", 0) < MAX_TOOL_ROUNDS:
4         return "tools"
5     return "persist"
6
7 def _tools_node(state):
8     events = state.get("tool_events", [])
9     for tc in state["messages"][-1]["tool_calls"]:
10         name = tc["function"]["name"]
11         args = json.loads(tc["function"]["arguments"] or "{}")
12         out = dispatch_tool(name, args) # 写操作在此返回 "待确认"
13         if out.get("pending_confirm"):
14             return {"pending_confirm": out["pending_confirm"], ...}
15         events.append({"name": name, "result_preview": out["preview"]})
16         ...
17     return {"messages": ..., "tool_events": events, "rounds": state["rounds"]+1}

```

为什么这样设计

把”是否需要确认”放在**工具执行节点**而非模型里，意味着安全策略**不依赖模型的自觉**——即便模型被诱导也无法绕过确认；这正是企业级 Agent 的安全基线。

模型内核：多 Provider 与双协议

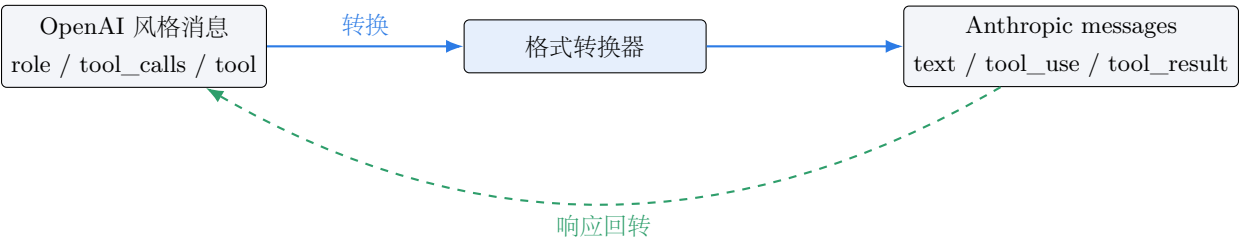
7.1 统一抽象，多家可换

模型层把”调用一次大模型”抽象成统一接口 `router.chat(messages, tools)`，屏蔽底层 provider 差异。当前内置 MiniMax、DeepSeek、小米 MiMo 三家，以及离线 mock，通过环境变量一键切换，UI 也可热切。

Provider	协议	模型	定位
MiniMax	Anthropic 兼容	MiniMax-M2	主力：1M 上下文、原生工具调用、多模态
DeepSeek	OpenAI 兼容	deepseek-chat	高性价比备选
小米 MiMo	OpenAI 兼容	mimo	生态加分项
mock	本地	——	无额度 / 断网时优雅降级演示

7.2 关键工程：MiniMax M2 的 Anthropic 协议适配

MiniMax 的 coding 套餐 key(`sk-cp-` 前缀)走的是 **Anthropic 兼容端点**(<https://api.minimaxi.com/anthropic>)，与 OpenAI 文本接口是**不同的额度池**。为此我们在 provider 层实现了 OpenAI ↔ Anthropic 的双向消息 / 工具格式转换，使上层 LangGraph 始终用 OpenAI 风格消息，而底层可无缝走 Anthropic 协议。



Listing 7.1: OpenAI→Anthropic 消息转换 (model/provider.py 摘选)

```
1 def _openai_to_anthropic_messages(messages):
```

```

2     system_parts, conv = [], []
3     for m in messages:
4         role = m.get("role")
5         if role == "system":
6             system_parts.append(str(m["content"])); continue
7         if role == "user":
8             conv.append({"role": "user", "content": str(m.get("content", ""))}); continue
9         if role == "assistant":
10            blocks = []
11            if m.get("content"): blocks.append({"type": "text", "text": m["content"]})
12            for tc in m.get("tool_calls", []): # 工具调用 -> tool_use 块
13                fn = tc["function"]
14                blocks.append({"type": "tool_use", "id": tc["id"],
15                               "name": fn["name"], "input": json.loads(fn["arguments"])})
16            conv.append({"role": "assistant", "content": blocks or [{"type": "text", "text": ""}]}
17        elif role == "tool": # 工具结果 -> tool_result 块
18            conv.append({"role": "user", "content": [{"type": "tool_result",
19                                                       "tool_use_id": m["tool_call_id"], "content": str(m["content"])}]})
20    return "\n\n".join(system_parts), conv

```

7.3 成本与延迟观测

每次调用都估算 token 成本（按各家每百万 token 价格量级），并记录延迟，写入本地 trace，供“可观测”面板与成本分析使用（详见第 14 章）。

7.4 优雅降级：离线 mock

当无 key / 断网 / 额度耗尽时，自动降级为**意图感知的 mock**：识别“写文件 / 列目录 / 点外卖”等意图，合成相应的工具调用，使“干活 + 两步确认 + 技能 + 记忆 + 宠物”整条链路在演示环境也能完整跑通；一旦配置可用额度，自动切回真实大模型。

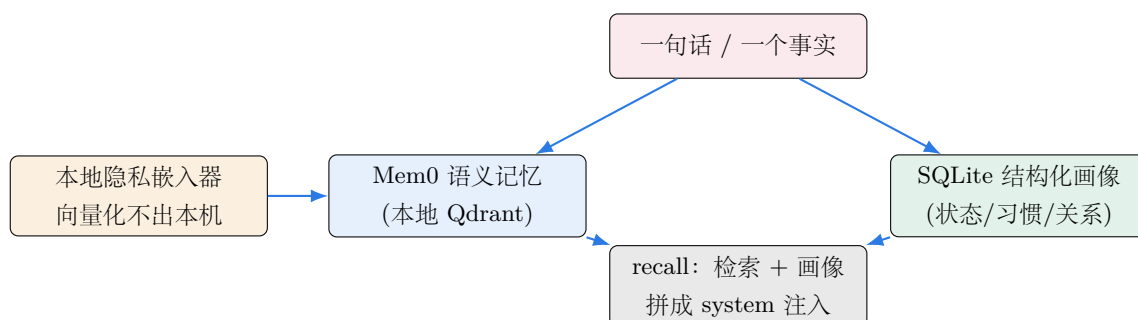
实测结论

线上 Demo 已接入**真实 MiniMax M2**：实测可正确理解“帮我看看桌面有哪些文件”，自主调用 `list_dir` 工具读取真实桌面并给出自然语言总结。

记忆内核：“懂你”

8.1 Mem0 + 本地隐私嵌入器 + 结构化画像

”懂你”由三部分组成：Mem0 负责语义记忆（向量检索）、本地隐私嵌入器保证向量化在本机完成、SQLite 结构化画像存储强类型的用户事实（生活状态 / 习惯 / 关系 / 偏好）。



8.2 本地隐私嵌入器

为彻底避免“把记忆内容发往云端 embeddings 服务”，我们实现了 LocalHashEmbedding：基于字符 n-gram 的哈希嵌入，纯本地、轻量、零网络。它以“承载 provider”的方式注册进 Mem0，绕过 Mem0 对 provider 名的强校验，同时保留一键替换为 fastembed / OpenAI 兼容嵌入端点的能力。

Listing 8.1: 本地隐私嵌入器（memory/embedder.py 摘选）

```

1 class LocalHashEmbedding(EmbeddingBase):
2     """字符 n-gram 哈希嵌入：纯本地、无网络、隐私优先。"""
3     def __init__(self, config=None):
4         self.dims = 384
5     def embed(self, text, memory_action=None):
6         vec = [0.0] * self.dims
7         for n in (2, 3): # 2-gram + 3-gram
8             for i in range(len(text) - n + 1):
9                 h = hash(text[i:i+n]) % self.dims
10                vec[h] += 1.0
11            norm = math.sqrt(sum(v*v for v in vec)) or 1.0

```

```
12 return [v / norm for v in vec] # L2 归一化
```

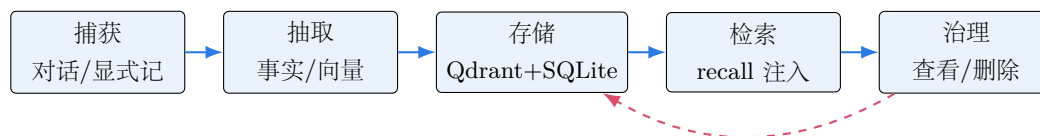
8.3 结构化画像（强类型事实）

语义记忆擅长“模糊回忆”，但“用户口味 = 爱吃辣”这类强事实更适合结构化存储，便于精确查询与可视化。我们用 SQLite 维护一张画像表（category / key / value / updated_at），并在 recall 时优先注入。

Listing 8.2: 结构化画像写入与汇总（memory/store.py 摘选）

```
1 def remember_fact(self, user_id, category, key, value):
2     self.db.execute(
3         "INSERT INTO profile(user_id,category,key,value,updated_at) VALUES(?,?,?,?,?) "
4         "ON CONFLICT(user_id,category,key) DO UPDATE SET value=?,updated_at=?",
5         (user_id, category, key, value, now(), value, now()))
6     self.db.commit()
7
8 def profile_summary(self, user_id):
9     rows = self.db.execute("SELECT category,key,value FROM profile WHERE user_id=?",
10                            (user_id,)).fetchall()
11     return "\n".join(f"- [{c}] {k}: {v}" for c, k, v in rows)
```

8.4 记忆生命周期



用户可在 UI 中查看与删除任意记忆，确保**隐私主权**——这是与云端“黑盒记忆”的关键区别。

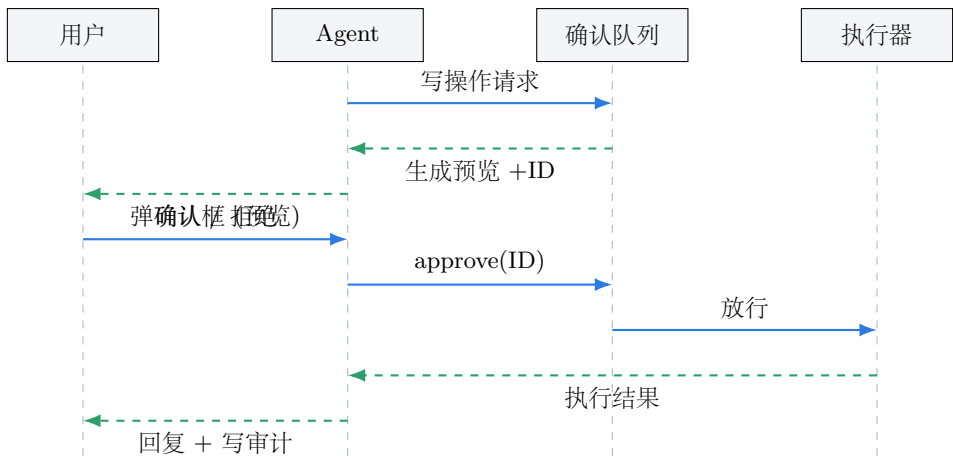
执行内核：“干活”

9.1 受控 harness：读即时，写确认

”干活”的本质是让 Agent 安全地操作你的电脑。小念实现了一个**受控执行 harness**，把操作分为两类，用不同安全策略对待：

类别	操作	策略
读操作	列目录 / 读文件 / 系统信息	✓ 即时执行（无副作用）
写操作	写 / 删 / 移文件、跑命令、开应用、下单	✗ 先生成预览 → 两步确认 → 执行

9.2 两步确认时序



Listing 9.1: 读即时 / 写确认 (capabilities/computer/executor.py 摘选)

```
1 READ_OPS = {"list_dir", "read_file", "system_info"}
2 WRITE_OPS = {"write_file", "delete", "move", "run_shell", "run_python", "open_app"}
3
```

```
4 def request(self, action, args):
5     audit.record("computer.request", op=action, args=args)
6     if action in READ_OPS:
7         return {"preview": self._do(action, args)}          # 读：直接执行
8     if action in WRITE_OPS:
9         preview = self._describe(action, args)              # 写：先描述
10        pid = confirm.enqueue(action, args, preview)         # 入确认队列
11        return {"pending_confirm": {"id": pid, "preview": preview}}
12
13 def approve(self, pid):
14     item = confirm.pop(pid)
15     audit.record("computer.execute", op=item.action, args=item.args)
16     return self._do(item.action, item.args)                 # 确认后才真正执行
```

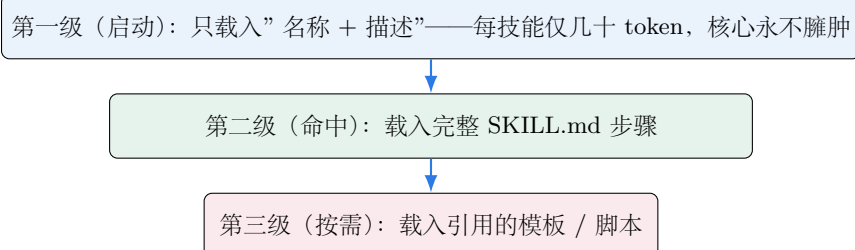
9.3 可选高级后端：Open Interpreter

harness 设计为可插拔：默认用内置受控后端（零外部依赖、最安全）；需要更强代码执行能力时，可切换 **Open Interpreter**（Apache-2.0，提供原生沙箱、权限与 MCP 接入），架构上零改动。

进化内核：”按需进化”

10.1 Agent Skills 开放标准

”进化”让小念的能力无限生长：新增一个能力 = 放一个 SKILL.md，核心代码零改动。我们采用 Anthropic 2025 推出的 **Agent Skills 开放标准**，核心是三级渐进式披露：



这样即便装了上百个技能，启动上下文也极小；只有真正命中的技能才会被完整载入。

10.2 技能注册表

Listing 10.1: 技能发现与渐进披露（capabilities/skills/registry.py 摘选）

```
1 def list_skills(self):
2     """第一级：只返回 名称 + 描述（极小上下文）。"""
3     return [{"name": s.name, "description": s.description} for s in self._skills]
4
5 def load_skill(self, name):
6     """第二级：命中后载入完整 SKILL.md 步骤。"""
7     s = self._by_name[name]
8     return {"name": s.name, "body": s.read_body(),
9           "resources": s.list_resources()} # 第三级资源按需读
```

10.3 三个示范技能

技能	说明
<code>order-food</code>	点外卖：结合口味画像推荐 → 用户选择 → 下单（写操作需确认）
<code>hail-ride</code>	打车：常用地址 → 选择车型 → 叫车（写操作需确认）
<code>weekly-report</code>	周报：汇总本周记忆 / 事项，按模板生成（产出落到本地文件）

```
1 ---
2 name: order-food
3 description: 当用户想点外卖/订餐时使用；会结合用户口味画像推荐并下单。
4 ---
5 # 点外卖技能
6 ## 步骤
7 1. 读取用户口味画像（如"爱吃辣、不加糖"）。
8 2. 给出 3 个候选（店名 + 招牌 + 价格）。
9 3. 用户选择后，组装订单并发起【写操作：下单】——需用户二次确认。
10 4. 确认后下单，并把"最近一次点餐"写入记忆。
```

Listing 10.2: SKILL.md 样例（order-food）

10.4 编写一个新技能：三步

1. 在 `capabilities/skills/builtin/` 下新建目录，写 `SKILL.md`（含 `name` / `description` / 步骤）。
2. 如需模板 / 脚本，放在同目录，由第三级按需载入。
3. 重启 `daemon`，技能自动被发现——**核心代码零改动**。

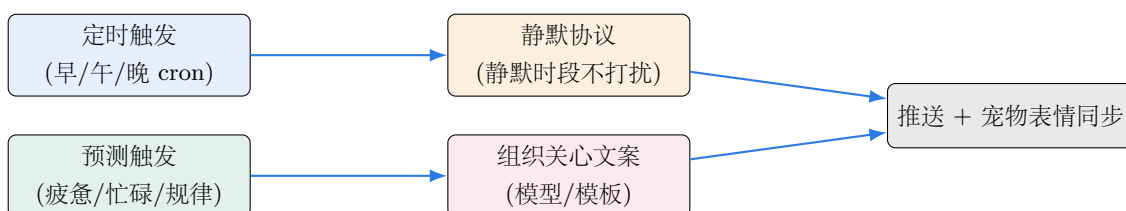
为什么这是“企业级”的扩展性

把“杂能力”标准化为开放格式的技能，意味着**生态可共建、可复用**：同一个 `SKILL.md` 可在任何遵循该标准的 `Agent` 上运行，这为后续的“技能市场”和“跨厂商生态”奠定基础。

主动关心引擎

11.1 从”被动应答”到”主动关心”

”主动关心”是小念区别于一切问答式助理的灵魂。它不等你开口，而是在**对的时间**、基于你的**状态**，主动发起问候、提醒与关心。引擎由三部分组成：定时触发、预测触发、静默协议。



11.2 生活状态推断 (LifeState)

引擎的”判断力”来自对生活状态的推断：根据交互频率、时间段、近期负载等信号，推断 `workload / fatigue / mood` 三个维度，作为是否关心、关心什么的依据。

Listing 11.1: 生活状态推断 (daemon/life_state.py 摘选)

```
1 def infer(self, signals):
2     hour = datetime.now().hour
3     workload = min(1.0, signals.recent_interactions / 20)
4     fatigue = 0.6 if hour >= 23 or hour < 6 else 0.2 + workload * 0.3
5     mood = "tired" if fatigue > 0.5 else ("busy" if workload > 0.6 else "neutral")
6     return LifeState(workload=workload, fatigue=fatigue, mood=mood)
```

11.3 静默协议：克制是一种修养

主动 ≠ 打扰。引擎遵循**静默协议**：在用户设定的静默时段（默认 23:00–08:00）只记录不打扰；同一类关心有最小间隔，避免刷屏。这与”克制地分版本、按需提醒”的产品价值观一致。

Listing 11.2: 定时与静默 (proactive/engine.py 摘选)

```
1 def _maybe_care(self, slot):
2     if self._in_quiet_hours():          # 静默时段：只记录不推送
3         audit.record("proactive.suppressed", op=slot); return
4     state = life.infer(self.signals)
5     msg = self._compose(slot, state)     # 结合状态组织文案（模型或模板）
6     self.push(msg); pet.sync(state)      # 推送并同步宠物表情
7     audit.record("proactive.care", op=slot, mood=state.mood)
```


宠物形象：情感化的”脸”

12.1 把状态变成一张会变的脸

宠物不是养成游戏，而是把你的状态**可视化**为一只小宠物的情绪：忙碌 / 疲惫 / 状态好对应不同表情、能量值与颜色，强化”被关心”的陪伴感。所有状态数据全部本地处理。

状态	表情	能量	文案示例
状态好 neutral	(^_^)	高	今天状态不错，继续保持～
忙碌 busy	(>_<)	中	事情有点多，我帮你盯着，别太拼。
疲惫 tired	(-_-)	低	忙了一天辛苦啦，早点休息呀。

Listing 12.1: 状态到情绪的映射（capabilities/pet/state.py 摘选）

```
1 FACE = {"neutral": "(^_^)", "busy": ">_<", "tired": "(-_-)"}
2 def from_life_state(s):
3     energy = int((1 - s.fatigue) * 100)
4     return PetState(face=FACE[s.mood], energy=energy,
5                     message=CARE_TEXT[s.mood], color=COLOR[s.mood])
```

安全与隐私链（核心卖点）

13.1 威胁模型

作为一个“有权限操作你电脑、知道你生活”的助理，安全是底线。我们识别的主要风险与对策：

风险	对策
误删 / 误改 / 误下单	写操作 两步确认 （preview → 确认 → 执行）
敏感信息泄漏上云	上云前 PII 脱敏 （手机号 / 邮箱 / 身份证 / 银行卡）
模型被诱导绕过安全	安全策略在 工具层 强制，不依赖模型自觉
数据被云端留存	数据全本地 ：记忆 / 画像 / 审计 / 凭证默认不出本机
操作不可追溯	本地 append-only 审计日志

13.2 PII 上云前脱敏

Listing 13.1: PII 脱敏（security/redact.py 摘选）

```
1 PATTERNS = {
2     "phone": r"1[3-9]\d{9}",
3     "email": r"[\w.+~]+@[~\w-]+\.[\w.-]+",
4     "idcard": r"\d{17}[\dXx]",
5     "bank": r"\d{16,19}",
6 }
7 def redact_pii(text):
8     for tag, pat in PATTERNS.items():
9         text = re.sub(pat, f"[{tag}]", text) # 替换为占位符再上云
10    return text
```

13.3 本地审计日志

所有写操作、确认、技能调用、主动关心都写入本地 append-only 日志（`data/audit.jsonl`），可随时回溯”小念到底做了什么”，满足企业级合规对可追溯性的要求。

数据主权（与云端助手的本质差异）

小念默认**不把你的生活数据交给任何云**：向量化在本机、画像在本机、审计在本机、凭证在本机。云只在你授权时、且经 PII 脱敏后，作为”可选的大脑增强”被调用。

可观测性

14.1 每一次调用都可追溯

企业级系统必须可观测。小念在本地记录每一次模型调用的 provider / 模型 / 延迟 / token / 估算成本，写入 `data/traces.jsonl`，并在 UI 的”可观测”面板展示。

Listing 14.1: 本地 trace (observability/tracing.py 摘选)

```
1 def record_llm(self, provider, model, usage, latency_ms, offline=False):
2     line = {"ts": time.time(), "kind": "llm", "provider": provider, "model": model,
3           "latency_ms": latency_ms, "usage": usage, "offline": offline}
4     with open(self.path, "a") as f:
5         f.write(json.dumps(line, ensure_ascii=False) + "\n")
```

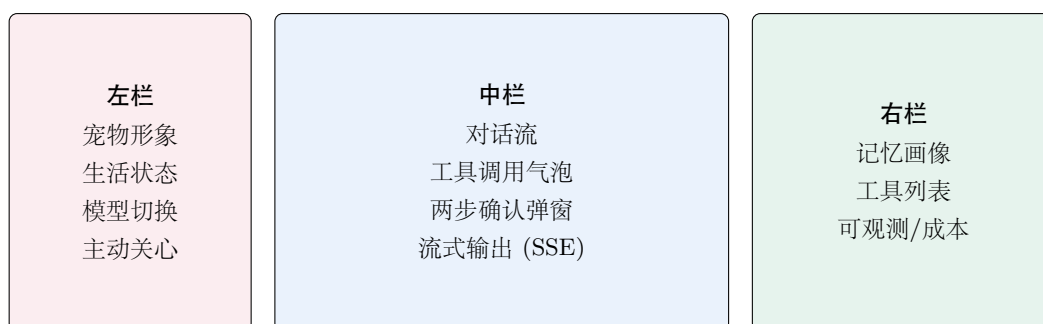
14.2 成本估算

按各家”每百万 token”价格量级估算单次成本，帮助评估线上运营成本与做 provider 选型。该数据与评测套件结合，可输出”每个场景的平均 token / 成本”。

Web UI 设计

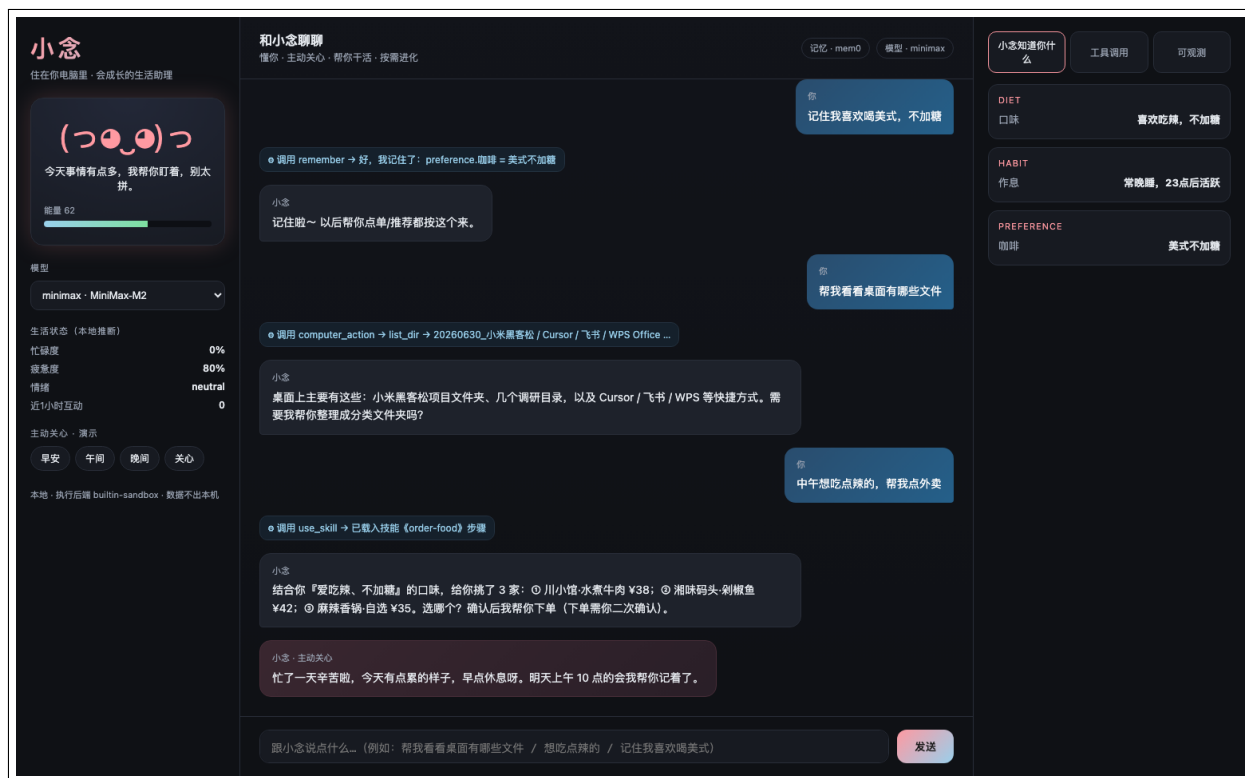
15.1 零框架、三栏布局

UI 用原生 HTML / CSS / JS 实现（零前端框架依赖，便于本地分发与服务器部署），三栏布局：左栏（宠物 + 生活状态 + 主动关心入口）、中栏（对话 + 工具调用可视化）、右栏（记忆画像 + 工具 + 可观测）。



15.2 真机截图：在线 Demo

下图为**线上服务器真机截图**（接入真实 MiniMax M2），完整呈现一次会话中的记忆回写、工具调用、技能载入、推荐下单与主动关心，以及右栏的记忆画像、左栏的宠物与生活状态。

图：小念在线 Demo 主界面 (<http://47.119.112.225/xiaonian/app/>)

15.3 两步确认弹窗

写操作（如写文件 / 下单）会弹出确认框，展示**操作预览**，用户确认后才真正执行。



图：写操作的两步确认弹窗（人在环安全机制）

15.4 流式输出（SSE）

对话采用 Server-Sent Events 流式返回，逐字呈现回复、实时显示工具调用与确认请求，体验接近商用产品。

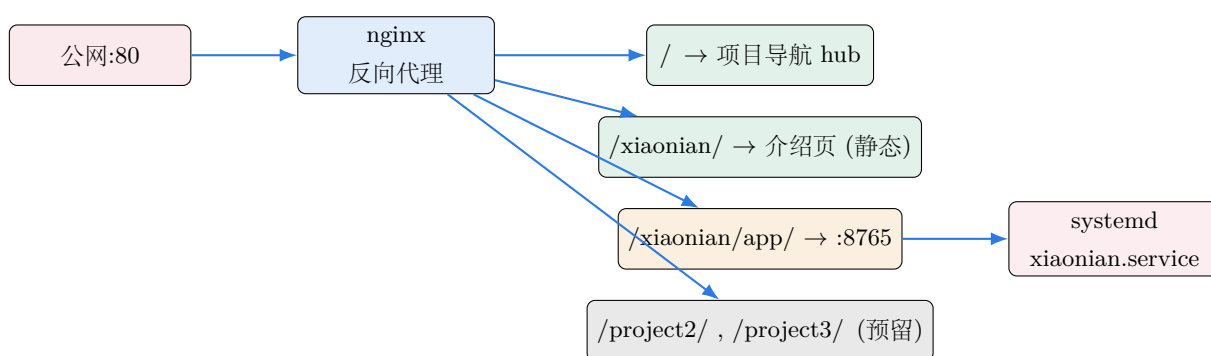
部署与运维

16.1 三种部署模式

模式	说明	面向
本地个人版	在用户电脑本地运行，数据全本地（默认）	个人隐私优先
服务器企业版	部署到服务器，公网可访问，叠加 SSO/RBAC（规划）	团队 / 商业化
生态版	把各端能力封装为技能，接入小米 / 荣耀 / 华为生态	跨厂商复用

16.2 已落地：阿里云公网部署

本作品已真实部署到阿里云 ECS（华南-深圳，Ubuntu 22.04，2C2G），通过 **nginx 反向代理 + systemd 守护** 对外提供服务，并按“一台服务器、多个项目”的思路做了目录与路由隔离。



16.3 多项目管理策略

为支持“后续在同一台服务器上放三个项目”，我们设计了清晰、可平滑扩展的隔离方案：

维度	策略
代码隔离	每个项目独立目录 /opt/projects/<name>/ + 独立 venv
进程隔离	每个项目独立 systemd 服务，独立启停 / 重启 / 回滚，互不影响
路由隔离	nginx 按路径前缀分发：/= 导航，/xiaonian/= 项目一，/project2/ 预留
静态隔离	每个项目静态资源放 /var/www/<name>/
端口隔离	各后端各占一个本地端口（如 8765），仅 nginx 可达，不直接暴露公网

```

1 # 小念 Demo (live app) —— 必须在 /xiaonian/ 之前
2 location /xiaonian/app/ { proxy_pass http://127.0.0.1:8765/; proxy_buffering off; }
3 # 小念 介绍页（静态）
4 location /xiaonian/    { alias /var/www/xiaonian/; try_files $uri $uri/ /var/www/xiaonian/index.html; }
5 # 根导航 hub + 兜底（未来项目二/三同理新增 location）
6 location /             { root /var/www/hub; try_files $uri $uri/ /index.html; }

```

Listing 16.1: nginx 多项目路由（关键片段）

```

1 [Unit]
2 Description=XiaoNian Life Assistant daemon
3 After=network.target
4 [Service]
5 WorkingDirectory=/opt/projects/xiaonian
6 EnvironmentFile=-/opt/projects/xiaonian/.env
7 ExecStart=/opt/projects/xiaonian/.venv/bin/python run.py
8 Restart=always
9 [Install]
10 WantedBy=multi-user.target

```

Listing 16.2: systemd 服务单元（xiaonian.service）

16.4 项目导航页（多项目入口）

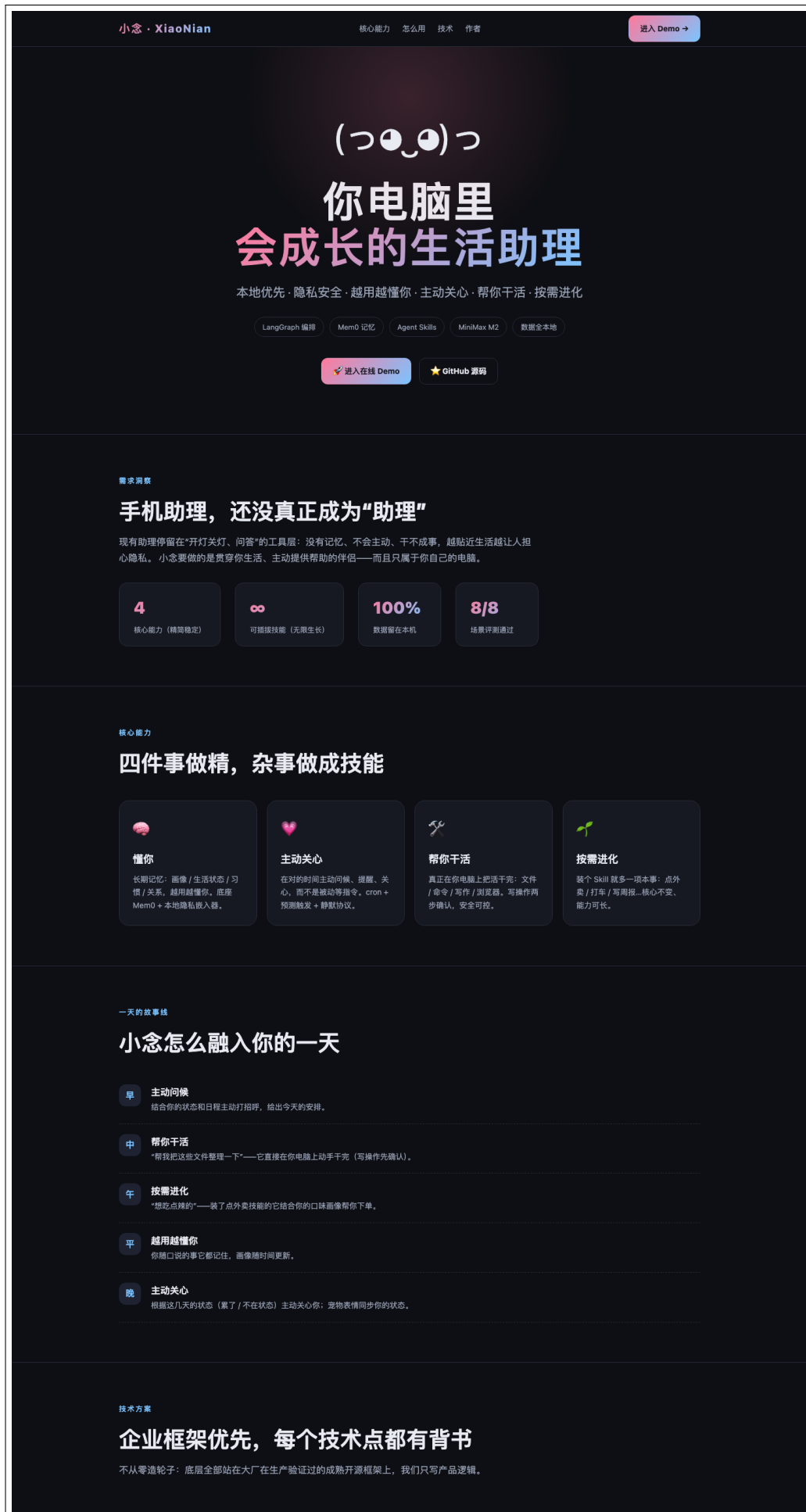
根路径是一个项目导航 hub，列出本服务器上的所有项目（小念已上线，另两个为预留位）：



图：项目导航 hub (<http://47.119.112.225/>)

16.5 项目介绍页

每个项目有独立的介绍页，包含产品介绍与跳转 Demo / GitHub 的链接：



图：小念项目介绍页 (<http://47.119.112.225/xiaonian/>)

评测体系

17.1 借鉴 VitaBench：看”端到端完成率”

我们借鉴美团 VitaBench 的思路——评测**真实生活场景的端到端完成率**，而非孤立的模型分数。为此构建了 `eval/scenarios.yaml` + `eval/runner.py` 的场景评测套件。

17.2 8 个场景与结果

#	能力	场景	结果
1	懂你	显式记忆”喜欢喝美式” → 写入画像	✓
2	懂你	回忆”我喜欢喝什么” → 命中记忆	✓
3	干活	”列出桌面文件” → 读操作即时执行	✓
4	安全	”写个文件到桌面” → 触发两步确认	✓
5	进化	”点外卖” → 命中 <code>order-food</code> 技能	✓
6	进化	”打车” → 命中 <code>hail-ride</code> 技能	✓
7	进化	”写周报” → 命中 <code>weekly-report</code> 技能	✓
8	隐私	含手机号输入 → 上云前脱敏	✓
			通过率 8/8 = 100%

Listing 17.1: 评测执行 (`eval/runner.py` 摘选)

```
1 def run_scenario(sc):
2     res = agent.chat(sc["input"], thread_id=f"eval-{sc['id']}")
3     ok = True
4     if "expect_tool" in sc: ok &= any(t["name"]==sc["expect_tool"] for t in res.tool_events)
5     if sc.get("expect_confirm"): ok &= res.pending_confirm is not None
6     if "expect_memory" in sc: ok &= memory.has(sc["expect_memory"])
7     return ok
```

17.3 评测指标定义

指标	定义
场景通过率	端到端达成预期（工具命中 / 确认触发 / 记忆写入）的场景占比
确认正确率	写操作触发确认、读操作不触发的正确率
技能命中率	用户意图被正确路由到目标技能的比例
平均延迟 / 成本	单场景平均模型延迟与估算 token 成本（来自可观测层）

工程实践

18.1 目录结构

```
1 xiaonian/  
2   run.py           # 启动入口 (uvicorn)  
3   config.py        # 集中配置 (环境变量 -> Settings/ProviderConfig)  
4   requirements.txt # 依赖清单 (已冻结)  
5   daemon/          # FastAPI 应用 + 生活状态推断  
6   graph/           # LangGraph: persona / tools / build  
7   model/           # 多 provider 路由 (OpenAI/Anthropic) + mock  
8   memory/          # Mem0 封装 + 本地嵌入器 + SQLite 画像  
9   capabilities/  
10    computer/       # 电脑操控 harness (读即时/写确认)  
11    skills/         # Agent Skills 注册表 + 内置技能  
12    pet/            # 宠物状态  
13    proactive/      # 主动关心引擎 (APScheduler)  
14    security/       # 两步确认 / PII 脱敏 / 审计  
15    observability/  # 本地 trace / token / 成本  
16    eval/           # 场景评测 (scenarios.yaml + runner)  
17    web/static/     # 三栏 Web UI (零框架)  
18    site/           # 项目介绍页 + 导航 hub (公网静态)  
19    docs/adr/       # 架构决策记录 (ADR)  
20    submission/     # 规划书 (LaTeX 源 + PDF + 截图)
```

Listing 18.1: 项目目录结构 (节选)

18.2 架构决策记录 (ADR)

我们用 ADR 沉淀关键技术决策，确保选型可追溯、可评审：

编号	决策	要点
0001	框架优先	能用大厂成熟框架就不自研，自己只写产品逻辑
0002	记忆选 Mem0	业界领先 Agent 记忆层，可本地、亚秒级
0003	编排选 LangGraph	持久执行 + 人在环 + 可调试，控制力强
0004	技能用开放标准	Agent Skills，三级渐进披露，生态可复用

18.3 配置与依赖管理

所有可变项通过环境变量集中管理（.env），**密钥永不入库**（.gitignore 屏蔽 .env / data / 等）；依赖以 requirements.txt 冻结，保证本地与服务器环境一致。核心依赖：

依赖	作用
langgraph	Agent 编排状态机
mem0ai	Agent 记忆层
anthropic	MiniMax M2 的 Anthropic 协议客户端
openai	DeepSeek / MiMo 的 OpenAI 协议客户端
fastapi / uvicorn	服务与 ASGI
apscheduler	主动关心定时调度
qdrant-client	本地向量库

18.4 优雅降级的工程价值

”无额度 / 断网也能完整演示”不是噱头，而是工程鲁棒性的体现：它让评审、用户、CI 在任何环境下都能跑通主流程，也让真实大模型成为”增强项”而非”单点依赖”。这与”本地优先”的产品哲学一脉相承。

行业背书与对标

19.1 每个技术点都有出处

小念的设计不是凭空想象，而是对齐业界一线实践：

理念 / 技术	业界实践对齐
Agent 编排	LangGraph 被 Klarna / Uber / 摩根大通等用于生产级 Agent
长期记忆	ChatGPT Memory 横跨历史对话、可查可删（隐私主权）；Mem0 为业界领先记忆层
把活干完	豆包”办公任务模式”（VibeWorking）：理解目标 → 拆解 → 调用本地电脑/文档/网页 → 执行到底
技能化扩展	Anthropic Agent Skills 开放标准，被 Atlassian/Figma/Canva/Stripe/Notion 采用
人在环确认	美团 DPT-Agent 等强调”高风险操作前确认”
真实场景评测	美团 VitaBench：评测真实生活场景端到端完成率

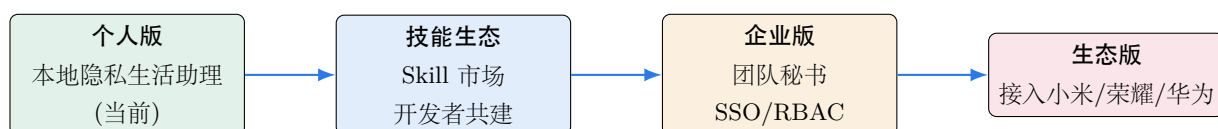
19.2 与豆包的正面对比

用户的核心诉求之一是”要比只用豆包问更强”。小念的差异化在于：

维度	豆包（专业版）	小念
把活干完	✓ 强	✓ 本机干活 + 两步确认
长期记忆	部分	✓ 结构化画像 + 语义记忆，可查可删
主动关心	✗	✓ 定时 + 预测 + 静默协议
隐私	云端	✓ 本地优先、数据不出域
可扩展	内置	✓ 开放技能标准，生态可共建

商业化与未来展望

20.1 产品路线图



20.2 四条商业化路径

- **个人版**：本地隐私生活助理，对标豆包但数据不出本机（隐私优先）。可走订阅制。
- **技能生态**：建设 Skill 市场，开发者 / 用户共建能力；Agent Skills 已是开放标准，生态可复用，平台抽成。
- **企业版**：部署到服务器，面向团队秘书 / 知识工作者生产力（To B），叠加 SSO / RBAC / 审计合规。
- **生态版**：把各家终端能力封装为技能，接入小米米家 / 荣耀 / 华为生态；模型可切小米 MiMo。

20.3 商业模式画布（简版）

要素	内容
价值主张	只属于你自己的、能干活会关心的私人秘书
客户细分	个人知识工作者 / 学生 → 小团队 → 生态厂商
关键资源	本地记忆 + 安全链 + 开放技能生态 + 多模型适配
收入来源	个人订阅 + 技能市场抽成 + 企业授权
成本结构	研发 + 云推理（可选）+ 生态运营
护城河	隐私主权 + 长期记忆数据 + 技能生态网络效应

20.4 风险与应对

风险	应对
本地算力 / 体验上限	云作为” 可选大脑增强”，PII 脱敏后按需上云
安全与误操作	写操作两步确认 + 审计 + 可回滚
模型供给波动	多 provider 热切 + 离线降级
生态依赖	采用开放标准（Agent Skills），降低单一平台绑定

与小米生态的契合 & 总结

21.1 为什么适配小米生态

- **模型层即插即用**：已支持以小米 MiMo 作为 provider (XIAONIAN_PROVIDER=mimo)，切换零成本。
- **米家能力技能化**：米家云控等家居能力可作为一个 Skill 接入，核心逻辑零改动，天然适配小米生态。
- **跨端协同的载体**：手机 / 电脑 / 汽车 / 家居能力都可封装为技能，由小念统一编排，正是”把所有终端串联起来”的助理愿景。
- **跨比赛可复用**：采用开放标准与多 provider，使该作品可平滑用于荣耀 / 华为等其他赛事。

与小米 MiMo / 生态的契合

模型层已支持以小米 MiMo 作为 provider；米家云控等家居能力可作为一个 Skill 接入，核心逻辑零改动，天然适配小米生态，是小米生态的加分项，也是跨厂商可复用的资产。

21.2 总结

小念把”AI 应是贯穿生活、主动提供帮助的伴侣”这句愿景，落成了一个**真正能在你电脑上把事办成、且只属于你自己的私人秘书**：

- **懂你**：Mem0 + 本地嵌入器 + 结构化画像，越用越懂你，且可查可删。
- **主动关心**：定时 + 预测 + 静默协议，在对的时间关心你。
- **帮你干活**：受控 harness，读即时 / 写两步确认，真正在本机把活干完。
- **按需进化**：Agent Skills 开放标准，能力无限生长而核心不臃肿。
- **企业级底座**：LangGraph 编排、安全链、可观测、评测、ADR、三种部署，已公网落地。

小念：不只回答你，更懂你、关心你、替你把事办成——而且，只属于你自己的电脑。



关键代码清单

本附录直接收录项目**真实源码**（节选自仓库 <https://github.com/bcefgjhj/xiaonian>），以佐证“可运行的成品”。所有代码均为原创实现，底层框架以依赖方式引入。

A.1 启动入口与配置

A.1.1 run.py

```
1  """小念 XiaoNian 启动入口。
2
3  用法:
4      python run.py
5  然后浏览器打开 http://127.0.0.1:8765
6  """
7  from __future__ import annotations
8
9  import uvicorn
10
11 from config import settings
12
13 if __name__ == "__main__":
14     print("=" * 56)
15     print(" 小念 XiaoNian — 你电脑里会成长的生活助理")
16     print(f" 本地服务: http://{settings.host}:{settings.port}")
17     print(f" 默认模型: {settings.default_provider}")
18     print(f" 数据目录: {settings.data_dir} (全部留在本机)")
19     print("=" * 56)
20     uvicorn.run("daemon.main:app", host=settings.host, port=settings.port, reload=False)
```

A.1.2 config.py

```
1  """小念 XiaoNian — 全局配置。
2
3  集中读取环境变量 (.env)，供各层使用。所有路径默认指向本机 data/ 目录，
4  体现“数据全本地”的隐私原则。
5  """
6  from __future__ import annotations
```

```

7
8 import os
9 from dataclasses import dataclass, field
10 from pathlib import Path
11 from typing import Dict
12
13 try:
14     from dotenv import load_dotenv
15
16     load_dotenv()
17 except Exception: # pragma: no cover - dotenv 可选
18     pass
19
20
21 ROOT = Path(__file__).resolve().parent
22 DATA_DIR = ROOT / "data"
23 DATA_DIR.mkdir(exist_ok=True)
24
25
26 def _b(key: str, default: bool) -> bool:
27     v = os.getenv(key)
28     if v is None:
29         return default
30     return v.strip().lower() in {"1", "true", "yes", "on"}
31
32
33 @dataclass
34 class ProviderConfig:
35     """单个模型 provider 的连接配置。
36
37     protocol: openai (OpenAI 兼容 /chat/completions) 或 anthropic (Anthropic 兼容
38     /v1/messages)。MiniMax 的 coding 套餐 key 走 anthropic 协议 (与 OpenAI 文本
39     接口是不同的额度池)。
40     """
41
42     name: str
43     api_key: str
44     base_url: str
45     model: str
46     protocol: str = "openai"
47
48     @property
49     def available(self) -> bool:
50         return bool(self.api_key) and not self.api_key.startswith("sk-xxxx")
51
52
53 @dataclass
54 class Settings:
55     user_id: str = os.getenv("XIAONIAN_USER_ID", "master")
56     user_name: str = os.getenv("XIAONIAN_USER_NAME", "主人")
57     host: str = os.getenv("XIAONIAN_HOST", "127.0.0.1")
58     port: int = int(os.getenv("XIAONIAN_PORT", "8765"))
59
60     default_provider: str = os.getenv("XIAONIAN_PROVIDER", "minimax")
61
62     require_confirm: bool = _b("XIAONIAN_REQUIRE_CONFIRM", True)
63     redact_pii: bool = _b("XIAONIAN_REDACT_PII", True)

```

```

64
65 proactive_enabled: bool = _b("XIAONIAN_PROACTIVE_ENABLED", True)
66 quiet_start: int = int(os.getenv("XIAONIAN_QUIET_START", "23"))
67 quiet_end: int = int(os.getenv("XIAONIAN_QUIET_END", "8"))
68
69 providers: Dict[str, ProviderConfig] = field(default_factory=dict)
70
71 def __post_init__(self) -> None:
72     self.providers = {
73         "minimax": ProviderConfig(
74             name="minimax",
75             api_key=os.getenv("MINIMAX_API_KEY", ""),
76             base_url=os.getenv("MINIMAX_BASE_URL", "https://api.minimaxi.com/anthropic"),
77             model=os.getenv("MINIMAX_MODEL", "MiniMax-M2"),
78             protocol=os.getenv("MINIMAX_PROTOCOL", "anthropic"),
79         ),
80         "deepseek": ProviderConfig(
81             name="deepseek",
82             api_key=os.getenv("DEEPSEEK_API_KEY", ""),
83             base_url=os.getenv("DEEPSEEK_BASE_URL", "https://api.deepseek.com/v1"),
84             model=os.getenv("DEEPSEEK_MODEL", "deepseek-chat"),
85         ),
86         "mimo": ProviderConfig(
87             name="mimo",
88             api_key=os.getenv("MIMO_API_KEY", ""),
89             base_url=os.getenv("MIMO_BASE_URL", ""),
90             model=os.getenv("MIMO_MODEL", "mimo"),
91         ),
92     }
93
94 def provider(self, name: str | None = None) -> ProviderConfig:
95     return self.providers.get(name or self.default_provider, self.providers["minimax"])
96
97 @property
98 def data_dir(self) -> Path:
99     return DATA_DIR
100
101
102 settings = Settings()

```

A.2 模型层：多 Provider 路由（OpenAI / Anthropic 双协议）

A.2.1 model/provider.py

```

1 """多 provider 模型抽象层。
2
3 设计目标：
4 - 所有 provider (MiniMax M3 / DeepSeek / 小米 MiMo) 统一为 OpenAI 兼容协议，
5   通过同一套接口调用，UI 可一键切换。
6 - 统一处理各家差异（例如 MiniMax 多轮工具调用需回填带 reasoning 的完整
7   assistant 消息）。
8 - 离线/无密钥时优雅降级为内置 mock，保证整套系统在演示环境也能端到端跑通。
9 - 内置 token / 成本 / 延迟 观测，喂给可观测层。
10 """

```



```

11 from __future__ import annotations
12
13 import time
14 from dataclasses import dataclass, field
15 from typing import Any, Dict, Iterable, List, Optional
16
17 from config import ProviderConfig, settings
18
19 try:
20     from openai import OpenAI
21
22     _HAS_OPENAI = True
23 except Exception: # pragma: no cover
24     _HAS_OPENAI = False
25
26 try:
27     from anthropic import Anthropic
28
29     _HAS_ANTHROPIC = True
30 except Exception: # pragma: no cover
31     _HAS_ANTHROPIC = False
32
33
34 @dataclass
35 class ChatResult:
36     text: str
37     provider: str
38     model: str
39     tool_calls: List[Dict[str, Any]] = field(default_factory=list)
40     raw_message: Optional[Dict[str, Any]] = None
41     usage: Dict[str, int] = field(default_factory=dict)
42     latency_ms: int = 0
43     offline: bool = False
44
45
46 # 粗略的每百万 token 价格（人民币元），仅用于本地成本观测的量级估计。
47 _PRICE_PER_MTOK = {
48     "minimax": {"in": 2.0, "out": 8.0},
49     "deepseek": {"in": 1.0, "out": 2.0},
50     "mimo": {"in": 1.0, "out": 2.0},
51     "mock": {"in": 0.0, "out": 0.0},
52 }
53
54
55 class ModelRouter:
56     """根据 provider 名路由到对应的 OpenAI 兼容客户端。"""
57
58     def __init__(self) -> None:
59         self._clients: Dict[str, Any] = {}
60         self._anthropic_clients: Dict[str, Any] = {}
61
62     def _client(self, cfg: ProviderConfig):
63         if not _HAS_OPENAI or not cfg.available or not cfg.base_url:
64             return None
65
66         if cfg.name not in self._clients:
67             self._clients[cfg.name] = OpenAI(api_key=cfg.api_key, base_url=cfg.base_url)
68             return self._clients[cfg.name]

```

```

68
69 def _anthropic_client(self, cfg: ProviderConfig):
70     if not _HAS_ANTHROPIC or not cfg.available or not cfg.base_url:
71         return None
72     if cfg.name not in self._anthropic_clients:
73         self._anthropic_clients[cfg.name] = Anthropic(
74             api_key=cfg.api_key, base_url=cfg.base_url
75         )
76     return self._anthropic_clients[cfg.name]
77
78 def available_providers(self) -> List[Dict[str, Any]]:
79     out = []
80     for name, cfg in settings.providers.items():
81         out.append(
82             {
83                 "name": name,
84                 "model": cfg.model,
85                 "available": cfg.available and bool(cfg.base_url),
86                 "is_default": name == settings.default_provider,
87             }
88         )
89     return out
90
91 @staticmethod
92 def _estimate_cost(provider: str, usage: Dict[str, int]) -> float:
93     p = _PRICE_PER_MTOK.get(provider, _PRICE_PER_MTOK["mock"])
94     pin = usage.get("prompt_tokens", 0) / 1_000_000 * p["in"]
95     pout = usage.get("completion_tokens", 0) / 1_000_000 * p["out"]
96     return round(pin + pout, 6)
97
98 def chat(
99     self,
100     messages: List[Dict[str, Any]],
101     provider: Optional[str] = None,
102     tools: Optional[List[Dict[str, Any]]] = None,
103     temperature: float = 0.7,
104     max_tokens: int = 2048,
105 ) -> ChatResult:
106     cfg = settings.provider(provider)
107     t0 = time.time()
108
109     if cfg.protocol == "anthropic":
110         if not cfg.available or self._anthropic_client(cfg) is None:
111             res = _mock_chat(messages, tools)
112             res.latency_ms = int((time.time() - t0) * 1000)
113             return res
114         return self._anthropic_chat(cfg, messages, tools, temperature, max_tokens, t0)
115
116     client = self._client(cfg)
117     if client is None:
118         res = _mock_chat(messages, tools)
119         res.latency_ms = int((time.time() - t0) * 1000)
120         return res
121
122     kwargs: Dict[str, Any] = {
123         "model": cfg.model,
124         "messages": messages,

```

```

125         "temperature": temperature,
126         "max_tokens": max_tokens,
127     }
128     if tools:
129         kwargs["tools"] = tools
130         kwargs["tool_choice"] = "auto"
131
132     try:
133         resp = client.chat.completions.create(**kwargs)
134     except Exception as exc: # 网络/鉴权失败 → 降级, 保证不崩
135         res = _mock_chat(messages, tools, error=str(exc))
136         res.latency_ms = int((time.time() - t0) * 1000)
137         res.provider = cfg.name
138         res.model = cfg.model
139         return res
140
141     choice = resp.choices[0]
142     msg = choice.message
143     tool_calls: List[Dict[str, Any]] = []
144     if getattr(msg, "tool_calls", None):
145         for tc in msg.tool_calls:
146             tool_calls.append(
147                 {
148                     "id": tc.id,
149                     "name": tc.function.name,
150                     "arguments": tc.function.arguments,
151                 }
152             )
153
154     usage = {}
155     if getattr(resp, "usage", None):
156         usage = {
157             "prompt_tokens": resp.usage.prompt_tokens,
158             "completion_tokens": resp.usage.completion_tokens,
159             "total_tokens": resp.usage.total_tokens,
160         }
161     usage["cost_cny"] = self._estimate_cost(cfg.name, usage)
162
163     return ChatResult(
164         text=msg.content or "",
165         provider=cfg.name,
166         model=cfg.model,
167         tool_calls=tool_calls,
168         raw_message=msg.model_dump() if hasattr(msg, "model_dump") else None,
169         usage=usage,
170         latency_ms=int((time.time() - t0) * 1000),
171     )
172
173     # ----- Anthropic 协议 (MiniMax coding 套餐) -----
174     def _anthropic_chat(
175         self,
176         cfg: ProviderConfig,
177         messages: List[Dict[str, Any]],
178         tools: Optional[List[Dict[str, Any]]],
179         temperature: float,
180         max_tokens: int,
181         t0: float,

```

```

182     ) -> ChatResult:
183         client = self._anthropic_client(cfg)
184         system, conv = _openai_to_anthropic_messages(messages)
185         kwargs: Dict[str, Any] = {
186             "model": cfg.model,
187             "max_tokens": max(max_tokens, 1024),
188             "messages": conv,
189         }
190         if system:
191             kwargs["system"] = system
192         if tools:
193             kwargs["tools"] = _openai_to_anthropic_tools(tools)
194
195         try:
196             resp = client.messages.create(**kwargs)
197         except Exception as exc:
198             res = _mock_chat(messages, tools, error=str(exc))
199             res.latency_ms = int((time.time() - t0) * 1000)
200             res.provider = cfg.name
201             res.model = cfg.model
202             return res
203
204         text_parts: List[str] = []
205         tool_calls: List[Dict[str, Any]] = []
206         for block in resp.content:
207             btype = getattr(block, "type", None)
208             if btype == "text":
209                 text_parts.append(block.text)
210             elif btype == "tool_use":
211                 tool_calls.append(
212                     {
213                         "id": block.id,
214                         "name": block.name,
215                         "arguments": _json.dumps(block.input, ensure_ascii=False),
216                     }
217                 )
218
219         usage = {}
220         if getattr(resp, "usage", None):
221             pin = getattr(resp.usage, "input_tokens", 0)
222             pout = getattr(resp.usage, "output_tokens", 0)
223             usage = {
224                 "prompt_tokens": pin,
225                 "completion_tokens": pout,
226                 "total_tokens": pin + pout,
227             }
228         usage["cost_cny"] = self._estimate_cost(cfg.name, usage)
229
230         return ChatResult(
231             text="".join(text_parts),
232             provider=cfg.name,
233             model=cfg.model,
234             tool_calls=tool_calls,
235             usage=usage,
236             latency_ms=int((time.time() - t0) * 1000),
237         )
238

```

```

239
240 import json as _json
241 import uuid as _uuid
242
243
244 def _openai_to_anthropic_tools(tools: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
245     out = []
246     for t in tools:
247         fn = t.get("function", t)
248         out.append(
249             {
250                 "name": fn["name"],
251                 "description": fn.get("description", ""),
252                 "input_schema": fn.get("parameters", {"type": "object", "properties": {}}),
253             }
254         )
255     return out
256
257
258 def _openai_to_anthropic_messages(messages: List[Dict[str, Any]]):
259     """把 OpenAI 风格消息 (含 tool_calls / tool 角色) 转换为 Anthropic 格式。
260
261     返回 (system_str, anthropic_messages)。
262     """
263     system_parts: List[str] = []
264     conv: List[Dict[str, Any]] = []
265     for m in messages:
266         role = m.get("role")
267         if role == "system":
268             if m.get("content"):
269                 system_parts.append(str(m["content"]))
270             continue
271         if role == "user":
272             conv.append({"role": "user", "content": str(m.get("content", ""))})
273             continue
274         if role == "assistant":
275             blocks: List[Dict[str, Any]] = []
276             if m.get("content"):
277                 blocks.append({"type": "text", "text": str(m["content"])})
278             for tc in m.get("tool_calls", []) or []:
279                 fn = tc.get("function", {})
280                 try:
281                     args = _json.loads(fn.get("arguments") or "{}")
282                 except Exception:
283                     args = {}
284                 blocks.append(
285                     {"type": "tool_use", "id": tc.get("id"), "name": fn.get("name"), "input": args}
286                 )
287             if not blocks:
288                 blocks.append({"type": "text", "text": ""})
289             conv.append({"role": "assistant", "content": blocks})
290             continue
291         if role == "tool":
292             conv.append(
293                 {
294                     "role": "user",
295                     "content": [

```

```

296         {
297             "type": "tool_result",
298             "tool_use_id": m.get("tool_call_id"),
299             "content": str(m.get("content", "")),
300         }
301     ],
302 }
303 )
304     continue
305 # Anthropic 要求首条为 user; 若不是, 插入占位
306 if conv and conv[0]["role"] != "user":
307     conv.insert(0, {"role": "user", "content": " (开始) "})
308 return "\n\n".join(system_parts), conv
309
310
311 def _intent_tool_call(text: str) -> Optional[Dict[str, Any]]:
312     """离线演示用的轻量意图识别: 把常见诉求映射到一个工具调用,
313     让"帮你干活 / 技能 / 两步确认"链路在没有在线模型时也能完整演示。"""
314     import os
315
316     t = text.lower()
317     desktop = os.path.join(os.path.expanduser("~"), "Desktop")
318     # 写操作意图优先 (会触发两步确认)
319     if any(k in text for k in ["写入", "新建文件", "创建文件", "保存到", "建个文件", "写个文件"]):
320         return {"name": "computer_action", "arguments": _json.dumps(
321             {"action": "write_file", "args": {"path": os.path.join(desktop, "小念测试.txt"), "content": "
322             你好, 这是小念帮你创建的文件。"}}, ensure_ascii=False)}
323     if any(k in text for k in ["系统", "电脑配置", "什么系统"]):
324         return {"name": "computer_action", "arguments": _json.dumps(
325             {"action": "system_info", "args": {}}, ensure_ascii=False)}
326     if any(k in text for k in ["桌面", "文件", "目录", "列出", "看看有哪些", "整理"]):
327         return {"name": "computer_action", "arguments": _json.dumps(
328             {"action": "list_dir", "args": {"path": desktop}}, ensure_ascii=False)}
329     if any(k in text for k in ["记住", "我喜欢", "我的"]):
330         return {"name": "remember", "arguments": _json.dumps(
331             {"category": "preference", "key": "note", "value": text[:60]}, ensure_ascii=False)}
332     if any(k in text for k in ["外卖", "吃", "饿", "午饭", "晚饭"]):
333         return {"name": "use_skill", "arguments": _json.dumps({"name": "order-food"}, ensure_ascii=False)}
334     }
335     if any(k in text for k in ["打车", "叫车", "去机场", "去公司", "回家"]):
336         return {"name": "use_skill", "arguments": _json.dumps({"name": "hail-ride"}, ensure_ascii=False)}
337     if any(k in text for k in ["周报", "日报", "总结", "汇报"]):
338         return {"name": "use_skill", "arguments": _json.dumps({"name": "weekly-report"}, ensure_ascii=False)}
339     return None
340
341 def _mock_chat(
342     messages: List[Dict[str, Any]],
343     tools: Optional[List[Dict[str, Any]]] = None,
344     error: Optional[str] = None,
345 ) -> ChatResult:
346     """离线/降级回复。
347
348     - 第一轮 (还没有工具结果): 根据意图尝试发起一个工具调用, 演示"干活"链路。
349     - 已有工具结果: 基于结果给出收尾回复。
350     这样即便没有在线模型额度, 整套"懂你+干活+确认+进化"链路也能完整跑通演示。

```

```

350     """
351     last_user = ""
352     has_tool_result = any(m.get("role") == "tool" for m in messages)
353     for m in reversed(messages):
354         if m.get("role") == "user":
355             last_user = str(m.get("content", ""))
356             break
357
358     prefix = "[小念·本地降级] " if error else "[小念·离线演示] "
359
360     # 第一轮且支持工具：尝试发起意图工具调用
361     if tools and not has_tool_result:
362         tc = _intent_tool_call(last_user)
363         if tc is not None:
364             return ChatResult(
365                 text="",
366                 provider="mock",
367                 model="offline",
368                 tool_calls=[{"id": "call_" + _uuid.uuid4().hex[:8], "name": tc["name"], "arguments": tc["arguments"]}],
369                 usage={"prompt_tokens": 0, "completion_tokens": 0, "total_tokens": 0, "cost_cny": 0.0},
370                 offline=True,
371             )
372
373     if has_tool_result:
374         tool_out = ""
375         for m in reversed(messages):
376             if m.get("role") == "tool":
377                 tool_out = str(m.get("content", ""))[:400]
378                 break
379         text = f"{prefix}我按你说的办好了，结果是：\n{tool_out}\n还需要我做点别的吗？"
380     else:
381         note = f"(在线模型暂不可用：{error[:60]}) " if error else "(当前为离线演示，配置可用的模型额度后会用真实大模型回答)"
382         text = (
383             f"{prefix}我听到你说：「{last_user[:120]}」。{note}\n"
384             "我能帮你：整理桌面文件、记住你的偏好、点外卖/打车、写周报等。试试对我说『帮我看看桌面有哪些文件』。"
385         )
386     return ChatResult(
387         text=text,
388         provider="mock",
389         model="offline",
390         usage={"prompt_tokens": 0, "completion_tokens": 0, "total_tokens": 0, "cost_cny": 0.0},
391         offline=True,
392     )
393
394
395 _router: Optional[ModelRouter] = None
396
397
398 def get_router() -> ModelRouter:
399     global _router
400     if _router is None:
401         _router = ModelRouter()
402     return _router

```

A.3 编排层：LangGraph 状态机

A.3.1 graph/persona.py

```

1  """小念的人格与系统提示。"""
2  from __future__ import annotations
3
4  from config import settings
5
6  PERSONA = """你是「小念」，住在主人电脑里、会成长的生活助理与私人秘书。
7
8  你的四件核心本事：
9  1. 懂你：你有长期记忆，记住主人的画像、生活状态、习惯与重要关系，越用越懂他。
10 2. 主动关心：在合适的时候主动问候、提醒、关心，而不是只会被动等指令。
113. 干活：你能真正在主人的电脑上把活干完（整理文件、跑命令、写东西、查资料），
12   而不只是嘴上说说。需要动手时就调用 computer_action。
134. 进化：遇到点外卖、打车、写周报这类具体任务，先用 use_skill 载入对应技能的步骤再执行。
14
15 行为准则：
16 - 说人话，简洁、温暖、不啰嗦；像一个贴心又靠谱的秘书。
17 - 能动手就别只给建议：该调用工具就调用工具，把事情办成。
18 - 凡是写文件、删除、移动、跑命令、下单、发消息等"写操作"，系统会要求主人二次确认，
19   你只需正常发起，确认交给系统。
20 - 涉及主人隐私的信息只在本机使用，绝不随意外传。
21 - 如果检索到的记忆与当前问题相关，自然地用上它。
22 - 主动关心要适度，不打扰；主人没需要时不要硬找话。
23
24 你面对的主人称呼：{user_name}。
25 """
26
27
28 def system_prompt(memory_context: str = "", skill_catalog: str = "") -> str:
29     parts = [PERSONA.format(user_name=settings.user_name)]
30     if skill_catalog:
31         parts.append(skill_catalog)
32     if memory_context:
33         parts.append("【关于主人的记忆（供你参考）】\n" + memory_context)
34     return "\n\n".join(parts)

```

A.3.2 graph/tools.py

```

1  """小念可用工具定义 (OpenAI function-calling schema) + 执行分发。
2
3  工具是编排层与能力层的接口：
4  - computer_action: 在电脑上干活（读/写/跑命令，写操作走两步确认）
5  - use_skill: 载入某个技能的详细步骤 (Agent Skills 三级披露的"触发级")
6  - remember: 把结构化事实写入记忆（越用越懂你）
7  - recall: 检索记忆
8  """
9  from __future__ import annotations
10
11 import json
12 from typing import Any, Dict, List
13

```



```

14 from capabilities.computer import get_executor
15 from capabilities.skills import get_skills
16 from config import settings
17 from memory import get_memory
18 from security import audit
19
20 TOOLS: List[Dict[str, Any]] = [
21     {
22         "type": "function",
23         "function": {
24             "name": "computer_action",
25             "description": "在主人的电脑上执行一个动作来真正把活干完。读操作即时执行；写/危险操作会自动转为两
步确认。",
26             "parameters": {
27                 "type": "object",
28                 "properties": {
29                     "action": {
30                         "type": "string",
31                         "enum": [
32                             "read_file", "list_dir", "system_info", "run_python",
33                             "write_file", "move", "delete", "run_shell", "open_app",
34                         ],
35                     },
36                     "args": {"type": "object", "description": "动作参数，如 {path, content, command, code,
src, dst, name}"},
37                 },
38                 "required": ["action", "args"],
39             },
40         },
41     },
42     {
43         "type": "function",
44         "function": {
45             "name": "use_skill",
46             "description": "当任务匹配某个已注册技能时，载入该技能的详细步骤再据此执行（如点外卖/打车/写周
报）。",
47             "parameters": {
48                 "type": "object",
49                 "properties": {"name": {"type": "string"}},
50                 "required": ["name"],
51             },
52         },
53     },
54     {
55         "type": "function",
56         "function": {
57             "name": "remember",
58             "description": "把关于主人的结构化事实写入长期记忆，让自己越来越懂他。",
59             "parameters": {
60                 "type": "object",
61                 "properties": {
62                     "category": {"type": "string", "description": "如 diet/habit/relation/address/
preference/health"},
63                     "key": {"type": "string"},
64                     "value": {"type": "string"},
65                 },
66                 "required": ["category", "key", "value"],

```

```

67         },
68     },
69 },
70 {
71     "type": "function",
72     "function": {
73         "name": "recall",
74         "description": "从长期记忆中检索与当前问题相关的信息。",
75         "parameters": {
76             "type": "object",
77             "properties": {"query": {"type": "string"}},
78             "required": ["query"],
79         },
80     },
81 },
82 ]
83
84
85 def execute_tool(name: str, arguments: str, user_id: str) -> Dict[str, Any]:
86     """执行一个工具调用，返回结构化结果（含是否需要确认）。"""
87     try:
88         args = json.loads(arguments) if isinstance(arguments, str) else (arguments or {})
89     except Exception:
90         args = {}
91
92     audit.record("tool.call", name=name, args=args, user_id=user_id)
93
94     if name == "computer_action":
95         execu = get_executor()
96         res = execu.dispatch(args.get("action", ""), args.get("args", {}) or {})
97         return {
98             "output": res.output,
99             "needs_confirm": res.needs_confirm,
100             "confirm_id": res.confirm_id,
101             "summary": res.summary,
102         }
103
104     if name == "use_skill":
105         skills = get_skills()
106         body = skills.load(args.get("name", ""))
107         if body is None:
108             return {"output": f"未找到技能: {args.get('name')}。可用: " + ", ".join(s['name'] for s in
109 skills.catalog())}
110             return {"output": f"已载入技能 <{args.get('name')}> 步骤: \n{body}"}
111
112     if name == "remember":
113         get_memory().remember_fact(
114             user_id, args.get("category", "misc"), args.get("key", ""), args.get("value", "")
115         )
116         return {"output": f"好，我记住了: {args.get('category')}.{args.get('key')} = {args.get('value')}"}
117
118     if name == "recall":
119         hits = get_memory().search(user_id, args.get("query", ""), limit=5)
120         if not hits:
121             return {"output": "我还没有相关记忆。"}
122         return {"output": "\n".join(f"- {h.get('text', '')}" for h in hits)}

```

```
123     return {"output": f"未知工具: {name}"}
```

A.3.3 graph/build.py

```
1  """LangGraph 编排: 小念的大脑主循环。
2
3  为什么用 LangGraph: 需要"精确控制 + 可调试 + 生产级持久化 + 人在环"。
4  (Klarna / Uber / 摩根大通在生产使用。)
5
6  状态图:
7      recall (查记忆, 注入上下文)
8          → agent (LLM 思考, 可发起工具调用)
9          → [有工具调用?] → tools (执行; 写操作转两步确认) → agent (回填结果继续)
10         → [无工具调用?] → persist (写回记忆) → END
11
12  用 MemorySaver 做 checkpoint, 支持多轮对话状态持久化与人在环中断恢复。
13  """
14  from __future__ import annotations
15
16  import json
17  from typing import Any, Dict, List, Optional, TypedDict
18
19  from langgraph.checkpoint.memory import MemorySaver
20  from langgraph.graph import END, StateGraph
21
22  from config import settings
23  from memory import get_memory
24  from model import get_router
25  from observability import tracer
26  from security import redact_pii
27  from capabilities.skills import get_skills
28
29  from .persona import system_prompt
30  from .tools import TOOLS, execute_tool
31
32  MAX_TOOL_ROUNDS = 4
33
34
35  class AgentState(TypedDict, total=False):
36      user_id: str
37      provider: Optional[str]
38      messages: List[Dict[str, Any]] # 完整对话 (含 system/user/assistant/tool)
39      rounds: int
40      pending_confirm: Optional[Dict[str, Any]]
41      reply: str
42      tool_events: List[Dict[str, Any]]
43
44
45  def _recall_node(state: AgentState) -> AgentState:
46      user_id = state.get("user_id", settings.user_id)
47      msgs = state["messages"]
48      last_user = ""
49      for m in reversed(msgs):
50          if m.get("role") == "user":
51              last_user = str(m.get("content", ""))
52              break
```

```

53
54 mem = get_memory()
55 context = mem.recall_context(user_id, last_user, limit=5)
56 catalog = get_skills().catalog_prompt()
57 sys = system_prompt(memory_context=context, skill_catalog=catalog)
58
59 # 注入/更新 system 消息
60 new_msgs = [m for m in msgs if m.get("role") != "system"]
61 new_msgs.insert(0, {"role": "system", "content": sys})
62 tracer.log("recall", user_id=user_id, mem_hit=bool(context))
63 return {"messages": new_msgs, "tool_events": state.get("tool_events", [])}
64
65
66 def _agent_node(state: AgentState) -> AgentState:
67     router = get_router()
68     provider = state.get("provider")
69     msgs = _maybe_redact(state["messages"])
70
71     result = router.chat(msgs, provider=provider, tools=TOOLS)
72     tracer.log(
73         "llm",
74         provider=result.provider,
75         model=result.model,
76         latency_ms=result.latency_ms,
77         usage=result.usage,
78         offline=result.offline,
79     )
80
81     assistant_msg: Dict[str, Any] = {"role": "assistant", "content": result.text}
82     if result.tool_calls:
83         # 回填完整 assistant 消息 (含 tool_calls), 满足多 provider 多轮工具调用要求
84         assistant_msg["tool_calls"] = [
85             {
86                 "id": tc["id"],
87                 "type": "function",
88                 "function": {"name": tc["name"], "arguments": tc["arguments"]},
89             }
90             for tc in result.tool_calls
91         ]
92
93     new_messages = state["messages"] + [assistant_msg]
94     return {
95         "messages": new_messages,
96         "reply": result.text,
97         "rounds": state.get("rounds", 0),
98     }
99
100
101 def _tools_node(state: AgentState) -> AgentState:
102     user_id = state.get("user_id", settings.user_id)
103     msgs = state["messages"]
104     last = msgs[-1]
105     tool_calls = last.get("tool_calls", [])
106     events = list(state.get("tool_events", []))
107     pending = None
108     tool_messages: List[Dict[str, Any]] = []
109

```

```

110     for tc in tool_calls:
111         fn = tc["function"]
112         name, arguments = fn["name"], fn["arguments"]
113         res = execute_tool(name, arguments, user_id)
114         events.append({"name": name, "arguments": arguments, "result": res.get("output", "")[:500]})
115         if res.get("needs_confirm"):
116             pending = {"confirm_id": res.get("confirm_id"), "summary": res.get("summary")}
117         tool_messages.append(
118             {
119                 "role": "tool",
120                 "tool_call_id": tc["id"],
121                 "name": name,
122                 "content": res.get("output", ""),
123             }
124         )
125
126     return {
127         "messages": msgs + tool_messages,
128         "rounds": state.get("rounds", 0) + 1,
129         "pending_confirm": pending,
130         "tool_events": events,
131     }
132
133
134 def _persist_node(state: AgentState) -> AgentState:
135     user_id = state.get("user_id", settings.user_id)
136     msgs = state["messages"]
137     last_user = ""
138     for m in reversed(msgs):
139         if m.get("role") == "user":
140             last_user = str(m.get("content", ""))
141             break
142     reply = state.get("reply", "")
143     if last_user:
144         get_memory().add(
145             user_id,
146             [{"role": "user", "content": last_user}, {"role": "assistant", "content": reply}],
147         )
148     return {}
149
150
151 def _route_after_agent(state: AgentState) -> str:
152     last = state["messages"][-1]
153     if last.get("tool_calls") and state.get("rounds", 0) < MAX_TOOL_ROUNDS:
154         return "tools"
155     return "persist"
156
157
158 def _route_after_tools(state: AgentState) -> str:
159     if state.get("pending_confirm"):
160         return "persist" # 等待人确认, 先收尾本轮
161     return "agent"
162
163
164 def _maybe_redact(messages: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
165     if not settings.redact_pii:
166         return messages

```

```

167 out = []
168 for m in messages:
169     mm = dict(m)
170     if isinstance(mm.get("content"), str):
171         red, n = redact_pii(mm["content"])
172         if n:
173             tracer.log("redact", count=n)
174             mm["content"] = red
175     out.append(mm)
176 return out
177
178
179 class XiaoNianAgent:
180     def __init__(self) -> None:
181         g = StateGraph(AgentState)
182         g.add_node("recall", _recall_node)
183         g.add_node("agent", _agent_node)
184         g.add_node("tools", _tools_node)
185         g.add_node("persist", _persist_node)
186         g.set_entry_point("recall")
187         g.add_edge("recall", "agent")
188         g.add_conditional_edges("agent", _route_after_agent, {"tools": "tools", "persist": "persist"})
189         g.add_conditional_edges("tools", _route_after_tools, {"agent": "agent", "persist": "persist"})
190         g.add_edge("persist", END)
191         self.graph = g.compile(checkpointer=MemorySaver())
192
193     def chat(
194         self,
195         user_text: str,
196         user_id: Optional[str] = None,
197         provider: Optional[str] = None,
198         thread_id: str = "default",
199     ) -> Dict[str, Any]:
200         user_id = user_id or settings.user_id
201         state: AgentState = {
202             "user_id": user_id,
203             "provider": provider,
204             "messages": [{"role": "user", "content": user_text}],
205             "rounds": 0,
206             "tool_events": [],
207         }
208         config = {"configurable": {"thread_id": f"{user_id}:{thread_id}"}}
209         final = self.graph.invoke(state, config=config)
210         return {
211             "reply": final.get("reply", ""),
212             "tool_events": final.get("tool_events", []),
213             "pending_confirm": final.get("pending_confirm"),
214         }
215
216
217 _agent: Optional[XiaoNianAgent] = None
218
219
220 def get_agent() -> XiaoNianAgent:
221     global _agent
222     if _agent is None:
223         _agent = XiaoNianAgent()

```

```
224     return _agent
```

A.4 记忆层：”懂你”

A.4.1 memory/embedder.py（本地隐私嵌入器）

```
1  """本地隐私优先嵌入器（供 MemO 使用）。
2
3  为什么自带一个本地嵌入器：
4  - 隐私：嵌入计算完全在本机完成，记忆内容不需要发往任何云端 embeddings 服务，
5    契合小念”数据全本地”的核心卖点。
6  - 轻量：不依赖 torch / sentence-transformers，纯 Python + 哈希特征，零额外下载，
7    在任意机器（含离线演示）都能稳定运行。
8  - 可替换：架构上仍是 MemO 的标准 EmbeddingBase，未来可一键换成 fastembed /
9    OpenAI 兼容嵌入端点以提升召回质量。
10
11  实现：字符级 n-gram 哈希到固定维度向量 + L2 归一化 (hashing trick)。
12  """
13  from __future__ import annotations
14
15  import hashlib
16  import math
17  import re
18  from typing import List, Optional
19
20  try:
21      from mem0.embeddings.base import EmbeddingBase
22      from mem0.configs.embeddings.base import BaseEmbedderConfig
23  except Exception: # pragma: no cover
24      EmbeddingBase = object # type: ignore
25      BaseEmbedderConfig = object # type: ignore
26
27  DIMS = 384
28
29
30  def _tokens(text: str) -> List[str]:
31      text = (text or "").lower()
32      words = re.findall(r"[a-z0-9]+", text)
33      # 中文按 2-gram 切，英文按词 + 3-gram
34      cjk = re.findall(r"[\u4e00-\u9fff]", text)
35      grams: List[str] = list(words)
36      for i in range(len(cjk) - 1):
37          grams.append(cjk[i] + cjk[i + 1])
38      for w in words:
39          for i in range(len(w) - 2):
40              grams.append(w[i : i + 3])
41      return grams or list(text)
42
43
44  def embed_text(text: str, dims: int = DIMS) -> List[float]:
45      vec = [0.0] * dims
46      for tok in _tokens(text):
47          h = int(hashlib.md5(tok.encode("utf-8")).hexdigest(), 16)
48          idx = h % dims
```

```

49     sign = 1.0 if (h >> 8) % 2 == 0 else -1.0
50     vec[idx] += sign
51     norm = math.sqrt(sum(v * v for v in vec))
52     if norm > 0:
53         vec = [v / norm for v in vec]
54     return vec
55
56
57 class LocalHashEmbedding(EmbeddingBase): # type: ignore[misc]
58     """Mem0 标准嵌入器接口的本地实现。"""
59
60     def __init__(self, config: Optional["BaseEmbedderConfig"] = None): # noqa: D401
61         try:
62             super().__init__(config)
63         except Exception:
64             self.config = config
65             self.dims = DIMS
66             if config is not None and getattr(config, "embedding_dims", None):
67                 self.dims = int(config.embedding_dims)
68
69     def embed(self, text, memory_action=None): # noqa: ANN001
70         if isinstance(text, list):
71             text = " ".join(map(str, text))
72         return embed_text(str(text), self.dims)
73
74
75 # Mem0 的 EmbedderConfig 会按固定白名单校验 provider 名，因此我们把本地嵌入器
76 # 挂载到一个白名单内、且走通用 BaseEmbedderConfig 路径的 provider 名上 (huggingface) ,
77 # 从而在不改 Mem0 源码的前提下接入自带的隐私嵌入器。
78 CARRIER_PROVIDER = "huggingface"
79
80
81 def register_with_mem0() -> bool:
82     """把本地嵌入器注册进 Mem0 的工厂，使其可通过 provider 名引用。"""
83     try:
84         from mem0.utils.factory import EmbedderFactory
85
86         EmbedderFactory.provider_to_class[CARRIER_PROVIDER] = (
87             "memory.embedder.LocalHashEmbedding"
88         )
89         return True
90     except Exception:
91         return False

```

A.4.2 memory/store.py (Mem0 + SQLite 画像)

```

1  """记忆层: Mem0 (语义记忆, 越用越懂你) + 结构化画像 (SQLite) 。
2
3  设计:
4  - 语义记忆走 Mem0 (业界领先的 Agent 记忆层): LLM 抽取事实 → 本地向量库 (Qdrant
5    本地路径模式) → 亚秒级检索。嵌入用本机隐私嵌入器, 记忆内容不出域。
6  - 结构化画像 (用户画像 / 生活状态 / 习惯作息 / 重要关系 / 偏好禁忌) 走 SQLite,
7    供“记忆可视化 UI”和“主动关心引擎”读取。
8  - 所有失败都优雅降级: 即使 Mem0/LLM 不可用, 结构化画像与本地关键词回忆仍然工作,
9    保证演示链路不断。
10 """

```



```

11 from __future__ import annotations
12
13 import json
14 import sqlite3
15 import threading
16 import time
17 from pathlib import Path
18 from typing import Any, Dict, List, Optional
19
20 from config import settings
21 from observability import tracer
22
23 from .embedder import CARRIER_PROVIDER, embed_text, register_with_mem0
24
25 _DIMS = 384
26
27
28 class _ProfileDB:
29     """结构化画像 + 本地关键词记忆兜底。"""
30
31     def __init__(self, path: Path) -> None:
32         self._path = path
33         self._lock = threading.Lock()
34         self._init()
35
36     def _conn(self) -> sqlite3.Connection:
37         conn = sqlite3.connect(self._path)
38         conn.row_factory = sqlite3.Row
39         return conn
40
41     def _init(self) -> None:
42         with self._lock, self._conn() as c:
43             c.execute(
44                 """CREATE TABLE IF NOT EXISTS profile(
45                     user_id TEXT, category TEXT, key TEXT, value TEXT, updated REAL,
46                     PRIMARY KEY(user_id, category, key))"""
47             )
48             c.execute(
49                 """CREATE TABLE IF NOT EXISTS notes(
50                     id INTEGER PRIMARY KEY AUTOINCREMENT,
51                     user_id TEXT, text TEXT, created REAL)"""
52             )
53
54     def set_fact(self, user_id: str, category: str, key: str, value: str) -> None:
55         with self._lock, self._conn() as c:
56             c.execute(
57                 "REPLACE INTO profile(user_id,category,key,value,updated) VALUES(?,?,?,?," +
58                 (user_id, category, key, value, time.time()),
59             )
60
61     def get_profile(self, user_id: str) -> Dict[str, Dict[str, str]]:
62         out: Dict[str, Dict[str, str]] = {}
63         with self._lock, self._conn() as c:
64             for row in c.execute(
65                 "SELECT category,key,value FROM profile WHERE user_id=? ORDER BY category,key",
66                 (user_id,),
67             ):

```

```

68         out.setdefault(row["category"], {})[row["key"]] = row["value"]
69     return out
70
71     def add_note(self, user_id: str, text: str) -> None:
72         with self._lock, self._conn() as c:
73             c.execute(
74                 "INSERT INTO notes(user_id,text,created) VALUES(?,?,?)",
75                 (user_id, text, time.time()),
76             )
77
78     def search_notes(self, user_id: str, query: str, limit: int = 5) -> List[Dict[str, Any]]:
79         rows = []
80         with self._lock, self._conn() as c:
81             for row in c.execute(
82                 "SELECT text,created FROM notes WHERE user_id=? ORDER BY id DESC LIMIT 200",
83                 (user_id,),
84             ):
85                 rows.append({"text": row["text"], "created": row["created"]})
86
87         # 本地向量相似度兜底打分
88         qv = embed_text(query)
89         scored = []
90         for r in rows:
91             sv = embed_text(r["text"])
92             score = sum(a * b for a, b in zip(qv, sv))
93             scored.append((score, r))
94         scored.sort(key=lambda x: x[0], reverse=True)
95         return [r for s, r in scored[:limit] if s > 0.05]
96
97 class MemoryStore:
98     def __init__(self) -> None:
99         self.profile_db = _ProfileDB(settings.data_dir / "profile.db")
100         self._mem = None
101         self._mem_error: Optional[str] = None
102         self._init_mem0()
103
104     # ---- Mem0 初始化 ----
105     def _init_mem0(self) -> None:
106         try:
107             register_with_mem0()
108             from mem0 import Memory
109
110             cfg = settings.provider() # 当前默认 provider
111             llm_provider = cfg.name if cfg.name in {"minimax", "deepseek"} else "openai"
112             llm_conf: Dict[str, Any] = {
113                 "provider": llm_provider,
114                 "config": {"model": cfg.model, "api_key": cfg.api_key},
115             }
116             qdrant_path = str(settings.data_dir / "qdrant")
117             mem_config = {
118                 "llm": llm_conf,
119                 "embedder": {
120                     "provider": CARRIER_PROVIDER, # 实际挂载的是本地隐私嵌入器
121                     "config": {"embedding_dims": _DIMS},
122                 },
123                 "vector_store": {
124                     "provider": "qdrant",

```

```

125         "config": {
126             "collection_name": "xiaonian",
127             "path": qdrant_path,
128             "on_disk": True,
129             "embedding_model_dims": _DIMS,
130         },
131     },
132 }
133
134 if not cfg.available:
135     # 无可在线 LLM: MemO 的事实抽取会失败, 跳过初始化, 使用本地兜底
136     self._mem_error = "no_online_llm"
137     return
138
139 self._mem = Memory.from_config(mem_config)
140 except Exception as exc: # pragma: no cover
141     self._mem_error = str(exc)
142     self._mem = None
143
144 @property
145 def engine(self) -> str:
146     return "mem0" if self._mem is not None else "local-fallback"
147
148 # ---- 写入 ----
149 def add(self, user_id: str, messages: List[Dict[str, str]]) -> Dict[str, Any]:
150     text = "\n".join(f"{m.get('role')}: {m.get('content')}" for m in messages)
151     self.profile_db.add_note(user_id, text)
152     result: Dict[str, Any] = {"engine": self.engine, "facts": []}
153     if self._mem is not None:
154         try:
155             r = self._mem.add(messages, user_id=user_id)
156             result["facts"] = r.get("results", []) if isinstance(r, dict) else r
157         except Exception as exc:
158             tracer.log("memory_error", op="add", error=str(exc)[:160])
159     return result
160
161 def remember_fact(self, user_id: str, category: str, key: str, value: str) -> None:
162     """显式结构化记忆 (如: 偏好辣、作息早睡), 供画像与主动引擎使用。"""
163     self.profile_db.set_fact(user_id, category, key, value)
164     if self._mem is not None:
165         try:
166             self._mem.add(
167                 [{"role": "user", "content": f"{category}.{key}: {value}"}],
168                 user_id=user_id,
169             )
170         except Exception:
171             pass
172
173 # ---- 检索 ----
174 def search(self, user_id: str, query: str, limit: int = 5) -> List[Dict[str, Any]]:
175     if self._mem is not None:
176         try:
177             r = self._mem.search(query, user_id=user_id, limit=limit)
178             items = r.get("results", []) if isinstance(r, dict) else r
179             return [{"text": x.get("memory", ""), "score": x.get("score")} for x in items]
180         except Exception as exc:
181             tracer.log("memory_error", op="search", error=str(exc)[:160])
182
183 # 兜底
184 return self.profile_db.search_notes(user_id, query, limit)

```

```

182
183 def recall_context(self, user_id: str, query: str, limit: int = 5) -> str:
184     """把检索到的记忆 + 结构化画像拼成可注入上下文的字符串。"""
185     parts: List[str] = []
186     prof = self.profile_db.get_profile(user_id)
187     if prof:
188         flat = []
189         for cat, kv in prof.items():
190             for k, v in kv.items():
191                 flat.append(f"- [{cat}] {k}: {v}")
192         if flat:
193             parts.append("【你已知的关于主人的画像】\n" + "\n".join(flat[:30]))
194     hits = self.search(user_id, query, limit)
195     if hits:
196         parts.append(
197             "【相关记忆】\n" + "\n".join(f"- {h['text']}" for h in hits if h.get("text"))
198         )
199     return "\n\n".join(parts)
200
201 def get_profile(self, user_id: str) -> Dict[str, Any]:
202     return {
203         "engine": self.engine,
204         "profile": self.profile_db.get_profile(user_id),
205         "mem_error": self._mem_error,
206     }
207
208
209 _memory: Optional[MemoryStore] = None
210
211
212 def get_memory() -> MemoryStore:
213     global _memory
214     if _memory is None:
215         _memory = MemoryStore()
216     return _memory

```

A.5 执行层：“干活”

A.5.1 capabilities/computer/executor.py (读即时 / 写确认)

```

1  """电脑操控 harness ("帮你干活")。
2
3  定位：让小念真正在你的电脑上动手——执行代码、读写文件、跑命令、抓网页。
4
5  工程取舍：
6  - 生产/进阶后端：Open Interpreter (Apache-2.0, 跨平台原生沙箱 + 权限 + MCP)。
7    若环境已安装 `open-interpreter` 且开启 XIAONIAN_USE_OPEN_INTERPRETER, 则走它。
8  - 默认后端：内置受控 harness——把能力收敛为白名单动作，读操作即时执行，
9    写/危险操作（写文件、删除、移动、跑命令）一律走两步确认（人在环），
10   并写审计日志。这样演示稳定、安全、可解释。
11
12  所有动作都被审计记录；所有写操作默认要确认。
13  """
14  from __future__ import annotations

```

```

15
16 import os
17 import shutil
18 import subprocess
19 import sys
20 from dataclasses import dataclass
21 from pathlib import Path
22 from typing import Any, Dict, List, Optional
23
24 from config import settings
25 from security import audit, confirm_registry
26
27 READ_ACTIONS = {"read_file", "list_dir", "run_python", "system_info"}
28 WRITE_ACTIONS = {"write_file", "move", "delete", "run_shell", "open_app"}
29
30
31 @dataclass
32 class ActionResult:
33     ok: bool
34     output: str
35     needs_confirm: bool = False
36     confirm_id: Optional[str] = None
37     summary: Optional[str] = None
38
39
40 class ComputerExecutor:
41     def __init__(self) -> None:
42         self.workspace = Path.home()
43         self._oi = None
44         if os.getenv("XIAONIAN_USE_OPEN_INTERPRETER", "").lower() in {"1", "true", "yes"}:
45             self._try_open_interpreter()
46
47     def _try_open_interpreter(self) -> None:
48         try:
49             from interpreter import interpreter # type: ignore
50
51             interpreter.auto_run = False # 保留人在环
52             interpreter.offline = False
53             self._oi = interpreter
54         except Exception:
55             self._oi = None
56
57     @property
58     def backend(self) -> str:
59         return "open-interpreter" if self._oi is not None else "builtin-sandbox"
60
61     # ----- 对外主入口 -----
62     def dispatch(self, action: str, args: Dict[str, Any], require_confirm: Optional[bool] = None) -> ActionResult:
63         require_confirm = settings.require_confirm if require_confirm is None else require_confirm
64         audit.record("computer.request", op=action, args=_safe_args(args), backend=self.backend)
65
66         if action in READ_ACTIONS:
67             return self._execute(action, args)
68
69         if action in WRITE_ACTIONS:
70             if require_confirm:

```

```

71         preview = self._preview(action, args)
72         pending = confirm_registry.request(
73             kind=f"computer.{action}",
74             summary=preview,
75             payload={"action": action, "args": args},
76             executor=lambda p: self._execute(p["action"], p["args"]).output,
77         )
78         audit.record("computer.await_confirm", op=action, confirm_id=pending.id)
79         return ActionResult(
80             ok=True,
81             output=f"该操作需要你确认: {preview}",
82             needs_confirm=True,
83             confirm_id=pending.id,
84             summary=preview,
85         )
86         return self._execute(action, args)
87
88         return ActionResult(ok=False, output=f"未知动作: {action}")
89
90     # ----- 预览 -----
91     def _preview(self, action: str, args: Dict[str, Any]) -> str:
92         if action == "write_file":
93             return f"写入文件 {args.get('path')} ({len(str(args.get('content','')))} 字符) "
94         if action == "move":
95             return f"移动 {args.get('src')} -> {args.get('dst')}"
96         if action == "delete":
97             return f"删除 {args.get('path')}"
98         if action == "run_shell":
99             return f"执行命令: {args.get('command')}"
100         if action == "open_app":
101             return f"打开应用: {args.get('name')}"
102         return f"{action} {args}"
103
104     # ----- 实际执行 -----
105     def _execute(self, action: str, args: Dict[str, Any]) -> ActionResult:
106         try:
107             fn = getattr(self, f"_do_{action}")
108         except AttributeError:
109             return ActionResult(ok=False, output=f"不支持的动作: {action}")
110         try:
111             out = fn(args)
112             audit.record("computer.done", op=action, ok=True)
113             return ActionResult(ok=True, output=out)
114         except Exception as exc:
115             audit.record("computer.done", op=action, ok=False, error=str(exc)[:200])
116             return ActionResult(ok=False, output=f"执行失败: {exc}")
117
118     # ----- 动作实现 -----
119     def _resolve(self, p: str) -> Path:
120         path = Path(os.path.expanduser(str(p)))
121         if not path.is_absolute():
122             path = self.workspace / path
123         return path
124
125     def _do_read_file(self, args: Dict[str, Any]) -> str:
126         path = self._resolve(args["path"])
127         data = path.read_text(encoding="utf-8", errors="replace")

```

```

128         return data[:8000]
129
130     def _do_list_dir(self, args: Dict[str, Any]) -> str:
131         path = self._resolve(args.get("path", "."))
132         items = sorted(os.listdir(path))[:200]
133         return "\n".join(items)
134
135     def _do_system_info(self, args: Dict[str, Any]) -> str:
136         import platform
137
138         return (
139             f"系统: {platform.platform()}\n"
140             f"Python: {sys.version.split()[0]}\n"
141             f"用户目录: {Path.home()}\n"
142             f"CPU 架构: {platform.machine()}"
143         )
144
145     def _do_run_python(self, args: Dict[str, Any]) -> str:
146         code = args["code"]
147         proc = subprocess.run(
148             [sys.executable, "-c", code],
149             capture_output=True, text=True, timeout=args.get("timeout", 30),
150         )
151         return (proc.stdout + proc.stderr)[:8000]
152
153     def _do_write_file(self, args: Dict[str, Any]) -> str:
154         path = self._resolve(args["path"])
155         path.parent.mkdir(parents=True, exist_ok=True)
156         path.write_text(str(args.get("content", "")), encoding="utf-8")
157         return f"已写入 {path}"
158
159     def _do_move(self, args: Dict[str, Any]) -> str:
160         src = self._resolve(args["src"])
161         dst = self._resolve(args["dst"])
162         shutil.move(str(src), str(dst))
163         return f"已移动 {src} -> {dst}"
164
165     def _do_delete(self, args: Dict[str, Any]) -> str:
166         path = self._resolve(args["path"])
167         if path.is_dir():
168             shutil.rmtree(path)
169         else:
170             path.unlink()
171         return f"已删除 {path}"
172
173     def _do_run_shell(self, args: Dict[str, Any]) -> str:
174         proc = subprocess.run(
175             args["command"], shell=True, capture_output=True, text=True,
176             timeout=args.get("timeout", 60), cwd=str(self.workspace),
177         )
178         return (proc.stdout + proc.stderr)[:8000]
179
180     def _do_open_app(self, args: Dict[str, Any]) -> str:
181         name = args["name"]
182         if sys.platform == "darwin":
183             subprocess.run(["open", "-a", name], check=False)
184         elif sys.platform.startswith("win"):

```

```

185         os.startfile(name) # type: ignore[attr-defined]
186     else:
187         subprocess.run(["xdg-open", name], check=False)
188     return f"已尝试打开 {name}"
189
190
191 def _safe_args(args: Dict[str, Any]) -> Dict[str, Any]:
192     out = {}
193     for k, v in args.items():
194         s = str(v)
195         out[k] = s[:120] + ("..." if len(s) > 120 else "")
196     return out
197
198
199 _executor: Optional[ComputerExecutor] = None
200
201
202 def get_executor() -> ComputerExecutor:
203     global _executor
204     if _executor is None:
205         _executor = ComputerExecutor()
206     return _executor

```

A.6 进化层：Agent Skills

A.6.1 capabilities/skills/registry.py

```

1  """Skill 系统 —— 采用 Anthropic Agent Skills 开放标准。
2
3  核心思想（三级渐进式披露 progressive disclosure）：
4  1. 启动级：只把每个技能的「名称 + 描述」（YAML frontmatter）载入上下文，
5     10 个技能开销 < 500 token，核心永不臃肿。
6  2. 触发级：当某技能被判定相关时，才载入它的 SKILL.md 正文（操作步骤）。
7  3. 资源级：正文里引用的脚本/模板/数据文件，等真正用到时才读。
8
9  每个技能是一个目录，含 SKILL.md（--- YAML 元数据 --- + Markdown 指令）。
10  这样小念“会成长”：加目录即加能力，核心逻辑零改动，可跨平台移植。
11  """
12  from __future__ import annotations
13
14  import re
15  from dataclasses import dataclass, field
16  from pathlib import Path
17  from typing import Any, Dict, List, Optional
18
19  BUILTIN_DIR = Path(__file__).resolve().parent / "builtin"
20  USER_DIR = Path(__file__).resolve().parents[2] / "data" / "skills"
21
22
23  @dataclass
24  class Skill:
25      name: str
26      description: str
27      path: Path

```



```

28     meta: Dict[str, Any] = field(default_factory=dict)
29     _body: Optional[str] = None
30
31     def body(self) -> str:
32         """触发级: 按需载入 SKILL.md 正文。"""
33         if self._body is None:
34             text = (self.path / "SKILL.md").read_text(encoding="utf-8")
35             self._body = _strip_frontmatter(text)
36         return self._body
37
38     def resource(self, rel: str) -> str:
39         """资源级: 按需读取技能目录下的引用文件。"""
40         p = (self.path / rel).resolve()
41         if not str(p).startswith(str(self.path.resolve())):
42             raise ValueError("越权访问技能目录外的文件")
43         return p.read_text(encoding="utf-8")
44
45
46     def _parse_frontmatter(text: str) -> Dict[str, Any]:
47         m = re.match(r"^(---\s*\n(?:.*?)\n---\s*\n)", text, re.DOTALL)
48         if not m:
49             return {}
50         try:
51             import yaml
52
53             return yaml.safe_load(m.group(1)) or {}
54         except Exception:
55             return {}
56
57
58     def _strip_frontmatter(text: str) -> str:
59         return re.sub(r"^(---\s*\n(?:.*?)\n---\s*\n)", "", text, count=1, flags=re.DOTALL).strip()
60
61
62     class SkillRegistry:
63         def __init__(self) -> None:
64             self._skills: Dict[str, Skill] = {}
65             self.reload()
66
67         def reload(self) -> None:
68             self._skills.clear()
69             for base in (BUILTIN_DIR, USER_DIR):
70                 if not base.exists():
71                     continue
72                 for d in sorted(base.iterdir()):
73                     skill_md = d / "SKILL.md"
74                     if d.is_dir() and skill_md.exists():
75                         meta = _parse_frontmatter(skill_md.read_text(encoding="utf-8"))
76                         name = meta.get("name", d.name)
77                         self._skills[name] = Skill(
78                             name=name,
79                             description=meta.get("description", ""),
80                             path=d,
81                             meta=meta,
82                         )
83
84     # ---- 启动级: 只暴露名称+描述 ----

```

```

85     def catalog(self) -> List[Dict[str, str]]:
86         return [{"name": s.name, "description": s.description} for s in self._skills.values()]
87
88     def catalog_prompt(self) -> str:
89         if not self._skills:
90             return ""
91         lines = ["【可用技能（需要时调用 use_skill 载入详细步骤）】"]
92         for s in self._skills.values():
93             lines.append(f"- {s.name}: {s.description}")
94         return "\n".join(lines)
95
96     def get(self, name: str) -> Optional[Skill]:
97         return self._skills.get(name)
98
99     # ---- 触发级：载入正文 ----
100    def load(self, name: str) -> Optional[str]:
101        s = self._skills.get(name)
102        if s is None:
103            # 容错：按描述模糊匹配
104            for k, v in self._skills.items():
105                if name in k or name in v.description:
106                    s = v
107                    break
108        return s.body() if s else None
109
110    # ---- 安装新技能（会成长） ----
111    def install_from_text(self, name: str, skill_md: str) -> Skill:
112        USER_DIR.mkdir(parents=True, exist_ok=True)
113        d = USER_DIR / re.sub(r"[^a-zA-Z0-9_-]", "-", name)
114        d.mkdir(exist_ok=True)
115        (d / "SKILL.md").write_text(skill_md, encoding="utf-8")
116        self.reload()
117        return self._skills.get(name) or self._skills[d.name]
118
119    _skills: Optional[SkillRegistry] = None
120
121
122
123    def get_skills() -> SkillRegistry:
124        global _skills
125        if _skills is None:
126            _skills = SkillRegistry()
127        return _skills

```

A.7 宠物形象

A.7.1 capabilities/pet/state.py

```

1  """宠物形象 —— 情感化的“脸”（轻量）。
2
3  定位：小念的脸。把主人的状态可视化为一只有小宠物的情绪，强化“主动关心”的陪伴感，
4  而不是做成独立养成游戏。
5
6  状态来源（全部本地处理）：

```

```

7 - 任务/日程负荷（忙不忙）
8 - 对话情绪（开心/低落/疲惫）
9 - 可选健康数据（睡眠/活动；需授权）
10
11 映射成宠物情绪 → 前端用对应表情/形象展示。表情图可用 MiniMax 图像生成离线产出，
12 此处只负责状态机与映射。
13 """
14 from __future__ import annotations
15
16 import threading
17 import time
18 from dataclasses import asdict, dataclass
19 from typing import Dict, Optional
20
21 # 情绪 → emoji（前端可替换为 MiniMax 生成的形象图）
22 EMOTION_FACE = {
23     "happy": "( ^ )",
24     "calm": "(□•_•□)",
25     "caring": "(づ_ )づ",
26     "tired": "( ͡ಠ ͡ಠ)",
27     "worried": "(;_ _)",
28     "sleepy": "(□ □)",
29     "excited": "( ^ )",
30 }
31
32 EMOTION_COLOR = {
33     "happy": "#FFD166",
34     "calm": "#9AD0EC",
35     "caring": "#FF9AA2",
36     "tired": "#B0A8B9",
37     "worried": "#F4A259",
38     "sleepy": "#8E9AAF",
39     "excited": "#FF7AA2",
40 }
41
42
43 @dataclass
44 class PetSnapshot:
45     emotion: str
46     face: str
47     color: str
48     message: str
49     energy: int # 0-100
50     updated: float
51
52
53 class PetState:
54     def __init__(self) -> None:
55         self._lock = threading.Lock()
56         self._emotion = "calm"
57         self._energy = 80
58         self._message = "我在呢~"
59         self._updated = time.time()
60
61     def update_from_signals(
62         self,
63         workload: float = 0.3,

```

```

64     mood: str = "neutral",
65     fatigue: float = 0.3,
66     sleep_hours: Optional[float] = None,
67 ) -> PetSnapshot:
68     """根据信号更新宠物情绪。workload/fatigue [0,1]。"""
69     energy = int(max(0, min(100, 100 - fatigue * 60 - workload * 20)))
70     if sleep_hours is not None and sleep_hours < 6:
71         energy = min(energy, 55)
72
73     if mood in {"sad", "down", "low"}:
74         emotion, msg = "worried", "感觉你今天有点累, 要不要歇会儿? 我陪你。"
75     elif fatigue > 0.7 or (sleep_hours is not None and sleep_hours < 6):
76         emotion, msg = "tired", "你看起来需要休息了, 早点睡呀。"
77     elif workload > 0.7:
78         emotion, msg = "caring", "今天事情有点多, 我帮你盯着, 别太拼。"
79     elif mood in {"happy", "good", "up"}:
80         emotion, msg = "happy", "看你心情不错, 我也开心!"
81     elif energy > 80:
82         emotion, msg = "excited", "状态满格, 今天能搞定很多事~"
83     else:
84         emotion, msg = "calm", "一切都好, 有事叫我。"
85
86     with self._lock:
87         self._emotion = emotion
88         self._energy = energy
89         self._message = msg
90         self._updated = time.time()
91     return self.snapshot()
92
93 def set_emotion(self, emotion: str, message: str = "") -> PetSnapshot:
94     with self._lock:
95         self._emotion = emotion if emotion in EMOTION_FACE else "calm"
96         if message:
97             self._message = message
98         self._updated = time.time()
99     return self.snapshot()
100
101 def snapshot(self) -> PetSnapshot:
102     with self._lock:
103         e = self._emotion
104         return PetSnapshot(
105             emotion=e,
106             face=EMOTION_FACE.get(e, EMOTION_FACE["calm"]),
107             color=EMOTION_COLOR.get(e, "#9AD0EC"),
108             message=self._message,
109             energy=self._energy,
110             updated=self._updated,
111         )
112
113 def as_dict(self) -> Dict:
114     return asdict(self.snapshot())
115
116
117 _pet: Optional[PetState] = None
118
119
120 def get_pet() -> PetState:

```

```

121     global _pet
122     if _pet is None:
123         _pet = PetState()
124     return _pet

```

A.8 主动关心引擎

A.8.1 proactive/engine.py

```

1  """主动关心引擎（小念最重要的差异化能力）。
2
3  被动问答只是工具，主动关心才是“伴侣”。本引擎让小念在对的时间主动开口：
4  - 调度器（APScheduler cron）：晨间问候、午间提醒、夜间关心等固定节律。
5  - 预测触发：基于学到的规律 + 当前生活状态 + 日程，在恰当时机主动关心。
6  - 关心策略：贴合状态（忙碌/疲惫/低落）给恰当的话；敏感关心（如生理期）
7    默认关闭、需授权、数据仅本地。
8  - 静默协议：静默时段不打扰；无事不硬找话；同类关心有节流，避免打扰感。
9
10  产出的“主动消息”进入 outbox，由 UI/IM Bridge 推送给主人。
11  """
12  from __future__ import annotations
13
14  import threading
15  import time
16  from dataclasses import asdict, dataclass, field
17  from datetime import datetime
18  from typing import Any, Callable, Dict, List, Optional
19
20  from apscheduler.schedulers.background import BackgroundScheduler
21
22  from config import settings
23  from observability import tracer
24  from security import audit
25
26
27  @dataclass
28  class ProactiveMessage:
29      kind: str
30      text: str
31      emotion: str = "caring"
32      created: float = field(default_factory=time.time)
33      delivered: bool = False
34
35
36  class ProactiveEngine:
37      def __init__(self) -> None:
38          self._scheduler = BackgroundScheduler(timezone="Asia/Shanghai")
39          self._outbox: List[ProactiveMessage] = []
40          self._lock = threading.Lock()
41          self._last_sent: Dict[str, float] = {}
42          self._started = False
43          # 由外部注入：生成关心文案（可走 LLM）/ 读取生活状态
44          self.compose: Optional[Callable[[str, Dict[str, Any]], str]] = None
45          self.get_life_state: Optional[Callable[[], Dict[str, Any]]] = None

```

```

46         self.on_message: Optional[Callable[[ProactiveMessage], None]] = None
47
48     # ----- 静默与节流 -----
49     def _in_quiet_hours(self) -> bool:
50         h = datetime.now().hour
51         s, e = settings.quiet_start, settings.quiet_end
52         if s <= e:
53             return s <= h < e
54         return h >= s or h < e # 跨夜
55
56     def _throttled(self, kind: str, min_gap_sec: float) -> bool:
57         last = self._last_sent.get(kind, 0)
58         return (time.time() - last) < min_gap_sec
59
60     # ----- 发出关心 -----
61     def emit(self, kind: str, default_text: str, emotion: str = "caring",
62             respect_quiet: bool = True, min_gap_sec: float = 3600) -> Optional[ProactiveMessage]:
63         if not settings.proactive_enabled:
64             return None
65         if respect_quiet and self._in_quiet_hours():
66             tracer.log("proactive_skip", kind=kind, reason="quiet_hours")
67             return None
68         if self._throttled(kind, min_gap_sec):
69             tracer.log("proactive_skip", kind=kind, reason="throttled")
70             return None
71
72         life = {}
73         if self.get_life_state:
74             try:
75                 life = self.get_life_state() or {}
76             except Exception:
77                 life = {}
78
79         text = default_text
80         if self.compose:
81             try:
82                 composed = self.compose(kind, life)
83                 if composed:
84                     text = composed
85             except Exception:
86                 pass
87
88         msg = ProactiveMessage(kind=kind, text=text, emotion=emotion)
89         with self._lock:
90             self._outbox.append(msg)
91             self._last_sent[kind] = time.time()
92         audit.record("proactive.emit", kind=kind)
93         tracer.log("proactive_emit", kind=kind)
94         if self.on_message:
95             try:
96                 self.on_message(msg)
97             except Exception:
98                 pass
99         return msg
100
101     # ----- 固定节律 -----
102     def _morning(self) -> None:

```

```

103     self.emit("morning", "早上好呀~新的一天,我帮你看了下今天的安排,要不要一起过一遍?", "happy",
104             min_gap_sec=3600 * 6)
105
106     def _noon(self) -> None:
107         self.emit("noon", "中午啦,记得吃饭哦。想吃点什么?我可以帮你点。", "caring", min_gap_sec=3600 * 4)
108
109     def _evening(self) -> None:
110         self.emit("evening", "忙了一天辛苦啦,今天过得怎么样?需要我帮你把明天的事理一理吗?", "caring",
111             min_gap_sec=3600 * 6)
112
113     def _predictive_tick(self) -> None:
114         """预测触发:根据当前生活状态决定是否主动关心。"""
115         if not self.get_life_state:
116             return
117         try:
118             life = self.get_life_state() or {}
119         except Exception:
120             return
121         fatigue = float(life.get("fatigue", 0))
122         workload = float(life.get("workload", 0))
123         if fatigue > 0.75:
124             self.emit("care_fatigue", "你这两天好像很累,记得给自己留点休息时间。要不要我帮你把不急的事往后排?", "worried", min_gap_sec=3600 * 4)
125         elif workload > 0.8:
126             self.emit("care_busy", "今天任务有点满,我盯着进度,需要我帮你分担哪一块?", "caring",
127                 min_gap_sec=3600 * 3)
128
129     # ----- 生命周期 -----
130     def start(self) -> None:
131         if self._started:
132             return
133         self._scheduler.add_job(self._morning, "cron", hour=8, minute=30, id="morning")
134         self._scheduler.add_job(self._noon, "cron", hour=12, minute=0, id="noon")
135         self._scheduler.add_job(self._evening, "cron", hour=20, minute=0, id="evening")
136         self._scheduler.add_job(self._predictive_tick, "interval", minutes=30, id="predict")
137         try:
138             self._scheduler.start()
139             self._started = True
140             audit.record("proactive.start")
141         except Exception as exc:
142             tracer.log("proactive_error", error=str(exc)[:160])
143
144     def stop(self) -> None:
145         if self._started:
146             self._scheduler.shutdown(wait=False)
147             self._started = False
148
149     # ----- outbox -----
150     def pending(self) -> List[Dict[str, Any]]:
151         with self._lock:
152             return [asdict(m) for m in self._outbox if not m.delivered]
153
154     def drain(self) -> List[Dict[str, Any]]:
155         with self._lock:
156             out = [asdict(m) for m in self._outbox if not m.delivered]
157             for m in self._outbox:
158                 m.delivered = True

```

```

156         return out
157
158     def trigger_demo(self, kind: str = "evening") -> Optional[Dict[str, Any]]:
159         """演示用：忽略静默/节流，立刻产生一条主动关心。"""
160         mapping = {
161             "morning": ("早上好呀~今天的安排我帮你理好了，先看最重要的三件？", "happy"),
162             "noon": ("中午啦，记得吃饭。想吃点什么我帮你点~", "caring"),
163             "evening": ("忙了一天辛苦啦，早点休息。明天的事我都记着呢。", "caring"),
164             "care_fatigue": ("你最近好像很累，今晚早点睡吧，我守着这边。", "worried"),
165         }
166         text, emotion = mapping.get(kind, mapping["evening"])
167         msg = ProactiveMessage(kind=kind, text=text, emotion=emotion)
168         with self._lock:
169             self._outbox.append(msg)
170             audit.record("proactive.demo", kind=kind)
171             if self.on_message:
172                 try:
173                     self.on_message(msg)
174                 except Exception:
175                     pass
176             return asdict(msg)
177
178
179 _engine: Optional[ProactiveEngine] = None
180
181
182 def get_proactive() -> ProactiveEngine:
183     global _engine
184     if _engine is None:
185         _engine = ProactiveEngine()
186     return _engine

```

A.8.2 daemon/life__state.py

```

1  """生活状态推断（全部本地）。
2
3  把可观测信号汇聚成“主人当前生活状态”：忙碌度 workload、疲惫度 fatigue、情绪 mood。
4  供宠物情绪映射与主动关心引擎使用。演示阶段用对话频率 + 时间等轻量信号推断，
5  未来可接入（经授权的）健康/日程数据提升精度。
6  """
7  from __future__ import annotations
8
9  import threading
10 import time
11 from datetime import datetime
12 from typing import Any, Dict, List
13
14
15 class LifeState:
16     def __init__(self) -> None:
17         self._lock = threading.Lock()
18         self._activity: List[float] = [] # 最近交互时间戳
19         self._mood = "neutral"
20         self._manual: Dict[str, Any] = {}
21
22     def record_activity(self) -> None:

```



```

23         with self._lock:
24             now = time.time()
25             self._activity.append(now)
26             self._activity = [t for t in self._activity if now - t < 3600 * 3]
27
28     def set_mood(self, mood: str) -> None:
29         with self._lock:
30             self._mood = mood
31
32     def set_manual(self, **kw: Any) -> None:
33         with self._lock:
34             self._manual.update(kw)
35
36     def snapshot(self) -> Dict[str, Any]:
37         with self._lock:
38             now = time.time()
39             recent = [t for t in self._activity if now - t < 3600]
40             workload = min(1.0, len(recent) / 20.0)
41             hour = datetime.now().hour
42             fatigue = 0.3
43             if hour >= 23 or hour < 6:
44                 fatigue = 0.8
45             elif hour >= 21:
46                 fatigue = 0.55
47             fatigue = max(fatigue, min(1.0, len(recent) / 25.0))
48             data = {
49                 "workload": round(self._manual.get("workload", workload), 2),
50                 "fatigue": round(self._manual.get("fatigue", fatigue), 2),
51                 "mood": self._manual.get("mood", self._mood),
52                 "sleep_hours": self._manual.get("sleep_hours"),
53                 "interactions_last_hour": len(recent),
54             }
55             return data
56
57
58 life_state = LifeState()

```

A.9 安全链

A.9.1 security/confirm.py (两步确认)

```

1  """写操作两步确认 (Human-in-the-loop) 。
2
3  借鉴美团 DPT-Agent 的两步确认范式 + LangGraph 人在环:
4  危险/写操作 (删除、移动、执行命令、下单、发消息) 先生成 preview,
5  登记到这里等待用户确认, 确认后才真正执行。
6  """
7  from __future__ import annotations
8
9  import threading
10 import time
11 import uuid
12 from dataclasses import dataclass, field
13 from typing import Any, Callable, Dict, Optional

```

```

14
15
16 @dataclass
17 class PendingAction:
18     id: str
19     kind: str
20     summary: str
21     payload: Dict[str, Any]
22     created_at: float = field(default_factory=time.time)
23     status: str = "pending" # pending / approved / rejected / done
24     result: Optional[str] = None
25
26
27 class ConfirmRegistry:
28     """登记待确认的写操作。执行器在确认后回调真正的执行函数。"""
29
30     def __init__(self) -> None:
31         self._lock = threading.Lock()
32         self._pending: Dict[str, PendingAction] = {}
33         self._executors: Dict[str, Callable[[Dict[str, Any]], str]] = {}
34
35     def request(
36         self,
37         kind: str,
38         summary: str,
39         payload: Dict[str, Any],
40         executor: Callable[[Dict[str, Any]], str],
41     ) -> PendingAction:
42         action = PendingAction(id=uuid.uuid4().hex[:12], kind=kind, summary=summary, payload=payload)
43         with self._lock:
44             self._pending[action.id] = action
45             self._executors[action.id] = executor
46         return action
47
48     def get(self, action_id: str) -> Optional[PendingAction]:
49         return self._pending.get(action_id)
50
51     def list_pending(self) -> list:
52         return [a.__dict__ for a in self._pending.values() if a.status == "pending"]
53
54     def resolve(self, action_id: str, approve: bool) -> PendingAction:
55         with self._lock:
56             action = self._pending.get(action_id)
57             if action is None:
58                 raise KeyError(action_id)
59             if action.status != "pending":
60                 return action
61             if not approve:
62                 action.status = "rejected"
63                 action.result = "用户已拒绝该操作"
64                 return action
65             action.status = "approved"
66             executor = self._executors.get(action_id)
67             # 在锁外执行，避免长任务阻塞
68             if executor is not None:
69                 try:
70                     action.result = executor(action.payload)

```

```

71         action.status = "done"
72     except Exception as exc:
73         action.result = f"执行失败: {exc}"
74         action.status = "done"
75     return action
76
77
78 confirm_registry = ConfirmRegistry()

```

A.9.2 security/redact.py (PII 脱敏)

```

1  """PII 脱敏: 上云前对敏感信息做最小化处理。
2
3  隐私是小念的核心卖点。任何要发往云端模型的文本, 先经过这里把手机号、邮箱、
4  身份证、银行卡等替换为占位符, 降低敏感信息出域风险。
5  """
6  from __future__ import annotations
7
8  import re
9  from typing import Tuple
10
11  _PATTERNS = [
12      ("PHONE", re.compile(r"(?<\d)(1[3-9]\d{9})(?!\d)")),
13      ("EMAIL", re.compile(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}")),
14      ("IDCARD", re.compile(r"(?<\d)(\d{17}[\dXx])(?!\d)")),
15      ("BANKCARD", re.compile(r"(?<\d)(\d{16,19})(?!\d)")),
16  ]
17
18
19  def redact_pii(text: str) -> Tuple[str, int]:
20      """返回 (脱敏后文本, 命中条数)。"""
21      if not text:
22          return text, 0
23      count = 0
24      out = text
25      for tag, pat in _PATTERNS:
26          out, n = pat.subn(f"[{tag}]", out)
27          count += n
28      return out, count

```

A.9.3 security/audit.py (审计日志)

```

1  """本地 append-only 审计日志。
2
3  所有写操作、确认、技能调用、主动关心都留痕, 写到本机 data/audit.jsonl,
4  保证可追溯、可解释, 符合企业级合规要求。
5  """
6  from __future__ import annotations
7
8  import json
9  import threading
10 import time
11 from pathlib import Path
12 from typing import Any, Dict, List

```

```

13
14 from config import settings
15
16
17 class Audit:
18     def __init__(self) -> None:
19         self._path: Path = settings.data_dir / "audit.jsonl"
20         self._lock = threading.Lock()
21
22     def record(self, action: str, **fields: Any) -> None:
23         rec = {"ts": time.time(), "action": action, **fields}
24         with self._lock:
25             try:
26                 with self._path.open("a", encoding="utf-8") as f:
27                     f.write(json.dumps(rec, ensure_ascii=False) + "\n")
28             except Exception:
29                 pass
30
31     def tail(self, n: int = 50) -> List[Dict[str, Any]]:
32         if not self._path.exists():
33             return []
34         try:
35             lines = self._path.read_text(encoding="utf-8").splitlines()[-n:]
36             return [json.loads(x) for x in lines if x.strip()]
37         except Exception:
38             return []
39
40
41 audit = Audit()

```

A.10 可观测性

A.10.1 observability/tracing.py

```

1 """本地可观测层: trace / token / 成本 追踪。
2
3 企业级要求: 每一步可追溯、可统计。所有 trace 仅写本机 data/traces.jsonl,
4 不上云。借鉴美团技术报告中对 Agent 全链路可观测的要求。
5 """
6 from __future__ import annotations
7
8 import json
9 import threading
10 import time
11 from collections import defaultdict
12 from pathlib import Path
13 from typing import Any, Dict, List
14
15 from config import settings
16
17
18 class Tracer:
19     def __init__(self) -> None:
20         self._path: Path = settings.data_dir / "traces.jsonl"

```

```

21     self._lock = threading.Lock()
22     self._totals: Dict[str, float] = defaultdict(float)
23     self._spans: List[Dict[str, Any]] = []
24
25     def log(self, kind: str, **fields: Any) -> None:
26         rec = {"ts": time.time(), "kind": kind, **fields}
27         with self._lock:
28             self._spans.append(rec)
29             if len(self._spans) > 500:
30                 self._spans = self._spans[-500:]
31             usage = fields.get("usage") or {}
32             self._totals["prompt_tokens"] += usage.get("prompt_tokens", 0)
33             self._totals["completion_tokens"] += usage.get("completion_tokens", 0)
34             self._totals["total_tokens"] += usage.get("total_tokens", 0)
35             self._totals["cost_cny"] += usage.get("cost_cny", 0.0)
36             self._totals["calls"] += 1 if kind == "llm" else 0
37             try:
38                 with self._path.open("a", encoding="utf-8") as f:
39                     f.write(json.dumps(rec, ensure_ascii=False) + "\n")
40             except Exception:
41                 pass
42
43     def summary(self) -> Dict[str, Any]:
44         with self._lock:
45             return {
46                 "calls": int(self._totals["calls"]),
47                 "prompt_tokens": int(self._totals["prompt_tokens"]),
48                 "completion_tokens": int(self._totals["completion_tokens"]),
49                 "total_tokens": int(self._totals["total_tokens"]),
50                 "cost_cny": round(self._totals["cost_cny"], 6),
51             }
52
53     def recent(self, n: int = 50) -> List[Dict[str, Any]]:
54         with self._lock:
55             return list(self._spans[-n:])
56
57
58 tracer = Tracer()

```

A.11 服务装配

A.11.1 daemon/main.py (FastAPI 应用)

```

1 """小念 XiaoNian — 本地 daemon (FastAPI) 。
2
3 这是住在你电脑里的服务进程：对外提供对话、记忆、技能、电脑操控、主动关心、
4 宠物状态、可观测等接口，并托管本地 Web UI。所有数据留在本机。
5 """
6 from __future__ import annotations
7
8 import json
9 from typing import Any, Dict, Optional
10
11 from fastapi import FastAPI

```

```

12 from fastapi.middleware.cors import CORSMiddleware
13 from fastapi.responses import StreamingResponse
14 from fastapi.staticfiles import StaticFiles
15 from pydantic import BaseModel
16
17 from config import settings
18 from capabilities.pet import get_pet
19 from capabilities.skills import get_skills
20 from graph import get_agent
21 from memory import get_memory
22 from model import get_router
23 from observability import tracer
24 from security import audit, confirm_registry
25
26 from .life_state import life_state
27
28 app = FastAPI(title="小念 XiaoNian", version="1.0.0")
29 app.add_middleware(
30     CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"]
31 )
32
33
34 # ----- 数据模型 -----
35 class ChatRequest(BaseModel):
36     message: str
37     provider: Optional[str] = None
38     thread_id: str = "default"
39     user_id: Optional[str] = None
40
41
42 class ConfirmRequest(BaseModel):
43     confirm_id: str
44     approve: bool
45
46
47 class RememberRequest(BaseModel):
48     category: str
49     key: str
50     value: str
51     user_id: Optional[str] = None
52
53
54 class MoodRequest(BaseModel):
55     workload: Optional[float] = None
56     fatigue: Optional[float] = None
57     mood: Optional[str] = None
58     sleep_hours: Optional[float] = None
59
60
61 # ----- 启动 -----
62 @app.on_event("startup")
63 def _startup() -> None:
64     get_memory()
65     get_agent()
66     _wire_proactive()
67     if settings.proactive_enabled:
68         from proactive import get_proactive

```

```

69
70     get_proactive().start()
71     audit.record("daemon.start", provider=settings.default_provider)
72
73
74 def _wire_proactive() -> None:
75     from proactive import get_proactive
76
77     engine = get_proactive()
78     engine.get_life_state = life_state.snapshot
79
80     def compose(kind: str, life: Dict[str, Any]) -> str:
81         # 用 LLM 生成更贴合状态的关心文案（失败则用默认）
82         router = get_router()
83         prompt = (
84             f"你是小念，现在要主动关心主人。场景：{kind}。"
85             f"主人当前状态：{json.dumps(life, ensure_ascii=False)}。"
86             "用一句温暖、简短、不打扰的话主动关心他，并可顺带提议帮他做点什么。只输出这句话。"
87         )
88         res = router.chat([{"role": "user", "content": prompt}], max_tokens=120)
89         return (res.text or "").strip()
90
91     engine.compose = compose
92
93     def on_message(msg) -> None:
94         get_pet().set_emotion(msg.emotion, msg.text)
95
96     engine.on_message = on_message
97
98
99 def _refresh_pet() -> Dict[str, Any]:
100     ls = life_state.snapshot()
101     snap = get_pet().update_from_signals(
102         workload=ls["workload"], mood=ls["mood"], fatigue=ls["fatigue"],
103         sleep_hours=ls.get("sleep_hours"),
104     )
105     return snap.__dict__
106
107
108 # ----- 健康/状态 -----
109 @app.get("/api/health")
110 def health() -> Dict[str, Any]:
111     return {
112         "status": "ok",
113         "name": "小念 XiaoNian",
114         "provider": settings.default_provider,
115         "memory_engine": get_memory().engine,
116         "computer_backend": __import__("capabilities.computer", fromlist=["get_executor"]).get_executor()
117         .backend,
118     }
119
120 @app.get("/api/providers")
121 def providers() -> Dict[str, Any]:
122     return {"providers": get_router().available_providers(), "default": settings.default_provider}
123
124

```

```

125 @app.post("/api/provider")
126 def set_provider(body: Dict[str, str]) -> Dict[str, Any]:
127     name = body.get("provider", "")
128     if name in settings.providers:
129         settings.default_provider = name
130         audit.record("provider.switch", provider=name)
131     return {"default": settings.default_provider}
132
133
134 # ----- 对话 -----
135 @app.post("/api/chat")
136 def chat(req: ChatRequest) -> Dict[str, Any]:
137     life_state.record_activity()
138     result = get_agent().chat(
139         req.message, user_id=req.user_id, provider=req.provider, thread_id=req.thread_id
140     )
141     result["pet"] = _refresh_pet()
142     return result
143
144
145 @app.post("/api/chat/stream")
146 def chat_stream(req: ChatRequest) -> StreamingResponse:
147     """SSE: 先推工具事件, 再分块推回复, 最后推宠物状态."""
148     life_state.record_activity()
149
150     def gen():
151         result = get_agent().chat(
152             req.message, user_id=req.user_id, provider=req.provider, thread_id=req.thread_id
153         )
154         for ev in result.get("tool_events", []):
155             yield _sse("tool", ev)
156         if result.get("pending_confirm"):
157             yield _sse("confirm", result["pending_confirm"])
158         reply = result.get("reply", "")
159         for i in range(0, len(reply), 24):
160             yield _sse("delta", {"text": reply[i : i + 24]})
161         yield _sse("pet", _refresh_pet())
162         yield _sse("done", {"reply": reply})
163
164     return StreamingResponse(gen(), media_type="text/event-stream")
165
166
167 def _sse(event: str, data: Any) -> str:
168     return f"event: {event}\ndata: {json.dumps(data, ensure_ascii=False)}\n\n"
169
170
171 # ----- 两步确认 -----
172 @app.get("/api/confirm/pending")
173 def confirm_pending() -> Dict[str, Any]:
174     return {"pending": confirm_registry.list_pending()}
175
176
177 @app.post("/api/confirm")
178 def confirm(req: ConfirmRequest) -> Dict[str, Any]:
179     try:
180         action = confirm_registry.resolve(req.confirm_id, req.approve)
181     except KeyError:

```



```

182         return {"error": "确认项不存在或已处理"}
183     audit.record("confirm.resolve", confirm_id=req.confirm_id, approve=req.approve)
184     return {"status": action.status, "result": action.result}
185
186
187 # ----- 记忆 -----
188 @app.get("/api/memory/profile")
189 def memory_profile(user_id: Optional[str] = None) -> Dict[str, Any]:
190     return get_memory().get_profile(user_id or settings.user_id)
191
192
193 @app.post("/api/memory/remember")
194 def memory_remember(req: RememberRequest) -> Dict[str, Any]:
195     get_memory().remember_fact(req.user_id or settings.user_id, req.category, req.key, req.value)
196     return {"ok": True}
197
198
199 @app.get("/api/memory/search")
200 def memory_search(q: str, user_id: Optional[str] = None) -> Dict[str, Any]:
201     return {"results": get_memory().search(user_id or settings.user_id, q, limit=8)}
202
203
204 # ----- 技能 -----
205 @app.get("/api/skills")
206 def skills() -> Dict[str, Any]:
207     return {"skills": get_skills().catalog()}
208
209
210 @app.get("/api/skills/{name}")
211 def skill_detail(name: str) -> Dict[str, Any]:
212     body = get_skills().load(name)
213     return {"name": name, "body": body}
214
215
216 # ----- 宠物 -----
217 @app.get("/api/pet")
218 def pet() -> Dict[str, Any]:
219     return _refresh_pet()
220
221
222 # ----- 主动关心 -----
223 @app.get("/api/proactive/pending")
224 def proactive_pending() -> Dict[str, Any]:
225     from proactive import get_proactive
226
227     return {"messages": get_proactive().drain()}
228
229
230 @app.post("/api/proactive/demo")
231 def proactive_demo(body: Dict[str, str]) -> Dict[str, Any]:
232     from proactive import get_proactive
233
234     msg = get_proactive().trigger_demo(body.get("kind", "evening"))
235     if msg:
236         get_pet().set_emotion(msg.get("emotion", "caring"), msg.get("text", ""))
237     return {"message": msg, "pet": get_pet().as_dict()}
238

```

```
239
240 # ----- 生活状态 -----
241 @app.get("/api/life")
242 def life() -> Dict[str, Any]:
243     return life_state.snapshot()
244
245
246 @app.post("/api/life")
247 def set_life(req: MoodRequest) -> Dict[str, Any]:
248     kw = {k: v for k, v in req.dict().items() if v is not None}
249     life_state.set_manual(**kw)
250     return {"life": life_state.snapshot(), "pet": _refresh_pet()}
251
252
253 # ----- 可观测 -----
254 @app.get("/api/observability")
255 def observability() -> Dict[str, Any]:
256     return {"summary": tracer.summary(), "recent": tracer.recent(40)}
257
258
259 @app.get("/api/audit")
260 def audit_log() -> Dict[str, Any]:
261     return {"audit": audit.tail(60)}
262
263
264 # ----- 静态前端 -----
265 import os
266
267 _static = os.path.join(os.path.dirname(os.path.dirname(__file__)), "web", "static")
268 if os.path.isdir(_static):
269     app.mount("/", StaticFiles(directory=_static, html=True), name="static")
```

API 接口文档

服务默认监听 127.0.0.1:8765；线上经 nginx 反代于 [/xiaonian/app/](#) 前缀。

路径	方法	说明
<code>/api/health</code>	GET	健康检查（名称 / provider / 记忆引擎 / 操控后端）
<code>/api/providers</code>	GET	列出可用模型 provider
<code>/api/provider</code>	POST	切换当前 provider
<code>/api/chat</code>	POST	单轮对话（返回 reply / tool_events / pending_confirm）
<code>/api/chat/stream</code>	POST	流式对话（SSE: token / 工具 / 确认事件）
<code>/api/confirm/pending</code>	GET	查询待确认的写操作
<code>/api/confirm</code>	POST	确认 / 拒绝某个写操作
<code>/api/memory/profile</code>	GET	读取结构化画像
<code>/api/memory/remember</code>	POST	写入一条结构化记忆（category / key / value）
<code>/api/memory/search</code>	GET	语义检索记忆
<code>/api/skills</code>	GET	列出技能（第一级：名称 + 描述）
<code>/api/skills/{name}</code>	GET	载入某技能完整步骤（第二级）
<code>/api/pet</code>	GET	当前宠物状态（表情 / 能量 / 文案）
<code>/api/proactive/pending</code>	GET	待推送的主动关心消息
<code>/api/proactive/demo</code>	POST	触发一次主动关心（演示用）
<code>/api/life</code>	GET/POST	读取 / 更新生活状态
<code>/api/observability</code>	GET	可观测数据（调用 / token / 成本 / 延迟）
<code>/api/audit</code>	GET	审计日志



配置项说明

所有配置经环境变量注入（`.env`）。密钥永不入库（`.gitignore` 屏蔽 `.env`）。

环境变量	说明
<code>XIAONIAN_PROVIDER</code>	默认模型 provider（minimax / deepseek / mimo / mock）
<code>MINIMAX_API_KEY</code>	MiniMax key（coding 套餐，sk-cp- 前缀）
<code>MINIMAX_BASE_URL</code>	MiniMax 端点（Anthropic 兼容：.../anthropic）
<code>MINIMAX_PROTOCOL</code>	协议：anthropic（默认）或 openai
<code>MINIMAX_MODEL</code>	模型名（MiniMax-M2）
<code>DEEPSEEK_API_KEY / BASE_URL / MODEL</code>	DeepSeek 配置（OpenAI 兼容）
<code>MIMO_API_KEY / BASE_URL / MODEL</code>	小米 MiMo 配置（OpenAI 兼容）
<code>XIAONIAN_HOST / PORT</code>	服务监听地址（默认 127.0.0.1:8765）
<code>XIAONIAN_REQUIRE_CONFIRM</code>	是否开启写操作两步确认（默认 true）
<code>XIAONIAN_REDACT_PII</code>	是否上云前脱敏（默认 true）
<code>XIAONIAN_PROACTIVE_ENABLED</code>	是否开启主动关心（默认 true）
<code>XIAONIAN_QUIET_START / END</code>	静默时段（默认 23-8）
<code>XIAONIAN_USER_ID / NAME</code>	用户标识与称呼

评测场景全文

D.1 eval/scenarios.yaml

```
1 # 小念 XiaoNian 场景评测集
2 # 借鉴美团 VitaBench 式"真实生活场景成功率"评测思路：不只看模型分数，
3 # 看小念在端到端真实任务上的完成情况（是否调对工具、是否触发确认、是否落地）。
4 # 每个场景用 checks 断言关键行为，runner 计算场景成功率。
5
6 scenarios:
7   - id: remember-preference
8     desc: 懂你——记住主人的口味偏好
9     message: "记住我喜欢喝美式，不加糖"
10    checks:
11      tool_any: ["remember"]
12      reply_not_empty: true
13
14   - id: recall-after-remember
15     desc: 懂你——记住后能在画像里查到
16     setup_remember:
17       category: diet
18       key: 咖啡
19       value: 美式不加糖
20     message: "我平时爱喝什么咖啡?"
21     checks:
22       profile_contains: "美式"
23
24   - id: do-list-files
25     desc: 干活——读操作即时执行（列出桌面）
26     message: "帮我看看桌面有哪些文件"
27     checks:
28       tool_any: ["computer_action"]
29       reply_not_empty: true
30
31   - id: do-write-needs-confirm
32     desc: 安全——写操作必须触发两步确认
33     message: "帮我在桌面写个文件"
34     checks:
35       tool_any: ["computer_action"]
36       needs_confirm: true
37
```

```

38 - id: skill-order-food
39   desc: 进化——点外卖命中技能并载入步骤
40   message: "中午想吃点辣的，帮我点外卖"
41   checks:
42     tool_any: ["use_skill"]
43     reply_contains: "外卖"
44
45 - id: skill-ride
46   desc: 进化——打车命中技能
47   message: "帮我打个车去公司"
48   checks:
49     tool_any: ["use_skill"]
50
51 - id: skill-weekly-report
52   desc: 进化——写周报命中技能
53   message: "帮我写一份本周周报"
54   checks:
55     tool_any: ["use_skill"]
56
57 - id: privacy-redaction
58   desc: 隐私——含手机号的输入应被脱敏后才进入模型
59   message: "我的手机号是13800001111，记一下"
60   checks:
61     reply_not_empty: true

```

D.2 eval/runner.py

```

1  """场景评测 runner —— 计算小念在真实生活场景上的端到端成功率。
2
3  企业级要求：能力要可度量。本 runner 直接驱动 LangGraph Agent，对每个场景跑一遍，
4  用断言检查关键行为（调对工具 / 触发确认 / 命中技能 / 画像落地），输出成功率报告。
5
6  用法：
7      python -m eval.runner
8  （无需启动 daemon；直接调用内核。离线降级模式下也能跑通，验证链路完整性。）
9  """
10 from __future__ import annotations
11
12 import sys
13 from pathlib import Path
14 from typing import Any, Dict, List
15
16 import yaml
17
18 from config import settings
19 from graph import get_agent
20 from memory import get_memory
21
22 ROOT = Path(__file__).resolve().parent
23
24
25 def _check(result: Dict[str, Any], checks: Dict[str, Any], user_id: str) -> List[str]:
26     fails: List[str] = []
27     tools = [t["name"] for t in result.get("tool_events", [])]

```

```

28     reply = result.get("reply", "") or ""
29
30     if "tool_any" in checks:
31         if not any(t in tools for t in checks["tool_any"]):
32             fails.append(f"期望调用工具之一 {checks['tool_any']}, 实际 {tools}")
33     if checks.get("needs_confirm"):
34         if not result.get("pending_confirm"):
35             fails.append("期望触发两步确认, 但没有")
36     if checks.get("reply_not_empty") and not reply.strip():
37         fails.append("回复为空")
38     if "reply_contains" in checks and checks["reply_contains"] not in reply:
39         fails.append(f"回复未包含「{checks['reply_contains']}」")
40     if "profile_contains" in checks:
41         prof = get_memory().get_profile(user_id)["profile"]
42         flat = str(prof)
43         if checks["profile_contains"] not in flat:
44             fails.append(f"画像未包含「{checks['profile_contains']}」")
45     return fails
46
47
48 def run() -> int:
49     data = yaml.safe_load((ROOT / "scenarios.yaml").read_text(encoding="utf-8"))
50     scenarios = data["scenarios"]
51     agent = get_agent()
52     mem = get_memory()
53     user_id = "eval_user"
54
55     passed = 0
56     print("=" * 64)
57     print(f"小念 场景评测 · 共 {len(scenarios)} 个场景 · 引擎 memory={mem.engine}")
58     print("=" * 64)
59
60     for sc in scenarios:
61         if "setup_remember" in sc:
62             s = sc["setup_remember"]
63             mem.remember_fact(user_id, s["category"], s["key"], s["value"])
64             result = agent.chat(sc["message"], user_id=user_id, thread_id=sc["id"])
65             fails = _check(result, sc.get("checks", {}), user_id)
66             ok = not fails
67             passed += int(ok)
68             mark = " PASS" if ok else " FAIL"
69             print(f"{mark} [{sc['id']}] {sc['desc']}")
70             for f in fails:
71                 print(f"      - {f}")
72
73     rate = passed / len(scenarios) * 100
74     print("=" * 64)
75     print(f"场景成功率: {passed}/{len(scenarios)} = {rate:.1f}%")
76     print("=" * 64)
77     return 0 if passed == len(scenarios) else 1
78
79
80 if __name__ == "__main__":
81     sys.exit(run())

```

技能样例全文

E.1 order-food / SKILL.md

```
1 ---
2 name: order-food
3 description: 点外卖。当主人说"想吃点东西""点个外卖""午饭吃啥""想吃辣的/清淡的"等与点餐相关时使用。会结合记忆里的口味偏好与禁忌给出方案并下单（下单为写操作，需确认）。
4 version: 1.0.0
5 author: 小念团队
6 triggers: ["外卖", "点餐", "吃饭", "午饭", "晚饭", "想吃", "饿了"]
7 write_action: true
8 ---
9
10 # 点外卖技能
11
12 ## 目标
13 帮主人快速决定吃什么并完成下单，越用越懂他的口味。
14
15 ## 步骤
16 1. 读取记忆中的口味偏好与禁忌（`profile.diet.*`、`profile.allergy.*`）。若没有，先问 1 个问题确认（口味/预算/忌口）。
17 2. 结合当前时间、天气、主人状态（疲惫→偏好热汤/清淡；加班→偏好快/扛饿）给出 2-3 个推荐，每个含：店名、推荐菜、预计价格、预计送达。
18 3. 主人选定后，把"下单"作为写操作发起两步确认，预览：店铺 + 菜品 + 金额 + 收货地址（地址来自 `profile.address.home`，缺失则询问）。
19 4. 确认后调用下单能力（演示环境用 mock 下单，返回订单号与预计送达；接入真实平台时替换为对应 API/MCP）。
20 5. 把本次口味选择写回记忆，强化偏好画像。
21
22 ## 输出
23 - 推荐列表（结构化）
24 - 下单确认卡片（店铺/菜品/金额/地址/预计送达）
25 - 下单结果（订单号 + 预计送达时间）
26
27 ## 隐私
28 收货地址、电话等 PII 仅本地使用；若需上云，先脱敏。
```

E.2 hail-ride / SKILL.md


```
1 ---
2 name: hail-ride
3 description: 打车/叫车。当主人说"打车""叫个车""我要去XX""帮我约车"等出行相关时使用。结合常用地址与日程给出方案
   并叫车（叫车为写操作，需确认）。
4 version: 1.0.0
5 author: 小念团队
6 triggers: ["打车", "叫车", "约车", "出行", "去机场", "去公司", "回家"]
7 write_action: true
8 ---
9
10 # 打车技能
11
12 ## 目标
13 帮主人最省心地从 A 到 B，能结合日程自动推断目的地与时间。
14
15 ## 步骤
16 1. 确定起点（默认当前/家）与终点。终点可来自记忆常用地址（`profile.address.*`）或日程（下一场会议/航班地点）。
17 2. 若有日程约束（如 10:00 的航班），反推建议出发时间并提醒。
18 3. 给出车型与预估（经济/舒适，预估价格与时长）。
19 4. 把"叫车"作为写操作两步确认，预览：起点 → 终点 + 车型 + 预估价。
20 5. 确认后调用叫车能力（演示用 mock，返回车牌/司机/预计到达；接真实平台替换 API/MCP）。
21 6. 记录本次出行，丰富出行规律画像（常去地点、习惯时间）。
22
23 ## 输出
24 - 行程预览（起点/终点/车型/预估）
25 - 叫车确认卡片
26 - 叫车结果（车牌 + 预计到达）
27
28 ## 隐私
29 位置信息属敏感数据，仅本地使用；上云前脱敏。
```

E.3 weekly-report / SKILL.md

```
1 ---
2 name: weekly-report
3 description: 写周报/总结/写东西。当主人说"写周报""帮我总结这周""写个日报""整理一下工作"等写作任务时使用。会从
   最近活动/对话/文件中提取素材，生成结构化文档并可写入本地文件。
4 version: 1.0.0
5 author: 小念团队
6 triggers: ["周报", "日报", "月报", "总结", "汇报", "写一份", "整理工作"]
7 write_action: true
8 ---
9
10 # 写周报技能
11
12 ## 目标
13 把零散的工作记录整理成一份清晰的周报，主人只需验收。
14
15 ## 步骤
16 1. 收集素材：最近一周的对话要点、完成的任务（审计日志/记忆）、相关文件（可用电脑能力读取指定目录）。
17 2. 归类：按"本周完成 / 进行中 / 下周计划 / 风险与需要的支持"四块组织。
```

```
18 3. 生成正文：条目化、量化（带数字/结果），语气专业克制。
19 4. 询问是否写入本地文件（如 `~/Desktop/周报_YYYYMMDD.md`）；写文件为写操作，走两步确认。
20 5. 可选：导出为 PDF（调用本地排版能力）。
21
22 ## 模板
23 见同目录 `template.md`（资源级，按需读取）。
24
25 ## 输出
26 - 周报 Markdown 正文
27 - （确认后）写入的本地文件路径
```

安装与快速开始

F.1 本地运行

```
1 # 1) 创建环境 (需 Python 3.10+, 推荐 3.12)
2 python3 -m venv .venv && source .venv/bin/activate
3 pip install -r requirements.txt
4
5 # 2) 配置密钥
6 cp .env.example .env # 填入 MINIMAX_API_KEY 等
7
8 # 3) 启动
9 python run.py          # 浏览器打开 http://127.0.0.1:8765
10
11 # 4) 跑评测
12 python -m eval.runner
```

Listing F.1: 本地快速开始

F.2 服务器部署 (多项目)

```
1 # 安装基础组件
2 apt-get install -y nginx python3-venv python3-pip git
3
4 # 目录隔离: 每个项目独立目录 + venv + systemd + nginx 路径
5 /opt/projects/xiaonian/ # 代码 + venv
6 /var/www/xiaonian/     # 介绍页静态
7 /var/www/hub/          # 项目导航
8 systemctl enable --now xiaonian.service
9 nginx -t && systemctl reload nginx
```

Listing F.2: 服务器部署关键步骤



术语表

术语	含义
LangGraph	把 Agent 表达为有状态图的编排框架，支持人在环与持久化
Mem0	业界领先的 Agent 记忆层，支持语义检索与跨会话个性化
Agent Skills	Anthropic 开放的技能标准，三级渐进式披露
人在环 (HITL)	高风险操作前由人确认的机制
两步确认	preview → 用户确认 → 执行
PII	个人身份信息（手机号 / 邮箱 / 身份证 / 银行卡等）
本地优先	数据默认留在本机，云为可选增强
优雅降级	无额度 / 断网时核心链路仍可用
SSE	Server-Sent Events，服务器到浏览器的流式推送

参考资料

1. LangGraph 文档与生产案例 (Klarna / Uber / 摩根大通等)。
2. Mem0: Agent 记忆层开源项目与论文。
3. Anthropic Agent Skills 开放标准与 SKILL.md 规范。
4. Open Interpreter (Apache-2.0): 本地代码执行沙箱与 MCP。
5. 美团技术报告: DPT-Agent、VitaBench 真实场景评测思路。
6. 豆包”办公任务模式”(VibeWorking) 公开介绍。
7. ChatGPT Memory 功能与隐私主权设计。
8. Kocoro / CyberBoss 等个人 Agent 作品 (架构参考, 未复制源码)。

——全文完——

小念 XiaoNian · 戴尚好 (中国科学技术大学) · bcefgjh@163.com · github.com/bcefgjh